How Far Can Interpretable Features Go?

Detecting Illicit Bitcoin Transactions on Elliptic++ Without Opaque Features

Marco Borghi
marco.borghi6@studio.unibo.it
Project for the course *Blockchain and Cryptocurrencies*, University of Bologna

November 2026

Abstract

The Elliptic dataset is the usual benchmark for finding illicit Bitcoin transactions, but its 165 features are anonymous, so models trained on it are hard to explain. We ask how much detection performance survives when those features are replaced by features with a clear meaning: 24 graph features that we compute from the transaction graph and 9 economic columns from Elliptic++. We train eight models on seven feature menus (132 final runs) and explain them with permutation importance, SHAP, Integrated Gradients and GNNExplainer. Named features keep most of the performance: on the test set the PR-AUC is 0.42–0.49 against 0.65 for the opaque features, and on the time steps before a dark market closes it is 0.79–0.81 against 0.91. For tree models, almost all of this comes from a few economic columns. The graph alone is weak (0.13–0.30), and added to the opaque features it gives a clear gain only to Logistic Regression. With the local features, tree models without edges beat every graph neural network. After the closure no feature set works: the PR-AUC of the best model falls from 0.91 to 0.04, because illicit transactions change shape and the rules the models learned stop applying.

Contents

1 Introduction **2**

2 Terms used in this report **3**

3 Data **4**

4 Features **6**

5 Models and training **11**

6 Results **12**

7 The closure of the dark market **20**

8 Inside the models **22**

9 Discussion **29**

10 Conclusion **30**

A Additional figures **30**

B Search spaces **33**

C Structure of the project **33**

1 Introduction

Bitcoin transactions are public, so money laundering can be studied directly on the transaction graph. The Elliptic dataset [1] is the usual benchmark for this task. It has 203,769 transactions, some of them labelled illicit or licit, and 165 features per transaction. The meaning of these features is not published. This is a real limit: a model can score a transaction, but nobody can say in plain words what it looked at.

The project proposal "Re-creation of Dataset Features" of this course suggests removing the original features and building new ones from the graph. Elliptic++ [2] adds a second source of readable information: 17 economic columns on the same transactions.

Research question. How much detection performance survives when the opaque features are replaced by features with a clear meaning, and which groups of features matter most?

What we did. We computed 27 graph features, each with a formula and a plain meaning (Section 4), and built seven feature *menus*, so that each comparison between two menus answers one precise question. We tuned and trained eight models on these menus, for 44 (menu, model) pairs and 132 final runs (Section 5), and compared them on a test set from the future of the training period (Section 6). Then we split the test set at the closure of a dark market reported by [1], because the first results pointed to it (Section 7), and we looked inside the models to see which columns they use and in which direction (Section 8).

Main findings.

- **Interpretable features keep most of the performance.** On the whole test set, the 33 named columns reach a PR-AUC of 0.42–0.49, against 0.65 for the best opaque menu. On the two time steps before the closure they reach 0.79–0.81 against 0.91.
- **A few economic columns do most of the work.** For the tree models, adding the graph features to the 9 economic columns changes nothing clearly. The economic family is the most important one in all seven models on the `transparent` menu.
- **The graph alone is weak, and it adds little.** Graph features alone give 0.13–0.30, which is 2.5 to 5.6 times the random classifier and far from 0.65. This puts numbers on the remark in [3]. Added to the opaque features, they give a clear gain only to Logistic Regression.
- **Trees win when local features are available.** XGBoost and Random Forest use no edges and still beat every graph neural network on the three menus with local features. Message passing helps a neural network on some menus, such as structure alone, and it clearly hurts on `transparent`.
- **No feature set survives the closure.** The PR-AUC of the best model falls from 0.914 to 0.042. Illicit transactions change shape after time step 43, and seven of the 44 pairs clearly rank them below licit ones.

2 Terms used in this report

The report uses a few terms in a precise way. Table 1 defines them, Table 2 lists the families and Table 3 lists the menus.

Table 1: Terms used in this report.

Term	Meaning
Transaction, node	A Bitcoin transaction. It is a node of the graph.
Edge	(u, v) means that v spends an output of u, so money flows from u to v.
Time step	A period of about two weeks. Each of the 49 time steps is one separate graph.
Class	Illicit (the positive class), licit, or unknown. Unknown means that the transaction has no label.
Evaluation row	A labelled transaction with the Elliptic++ data. Only these rows enter the loss and the metrics.
Opaque feature	A feature of the original Elliptic dataset. Its meaning is not published.
Transparent feature	A feature with a known meaning: computed by us from the graph, or an economic column.
Family	A group of columns with the same origin and purpose: LF, AF, A, B, C or D (Table 2).
Menu	The set of columns given to a model (Table 3).
Pair	One menu and one model, for example `transparent` and XGBoost. There are 44 pairs.
Triple	One pair and one seed. Each pair has seeds 0, 1 and 2, so there are 132 triples.
Prevalence	The share of illicit transactions among the evaluation rows. It is also the PR-AUC of a random classifier.
PR-AUC	Average precision: $\sum_k (R_k - R_{k-1}) P_k$, where P_k and R_k are precision and recall when the k highest scores are flagged. It is our main metric.
ROC-AUC	The probability that a random illicit transaction gets a higher score than a random licit one. 0.5 means random. It does not depend on the prevalence.
Clear difference	A difference whose 95% interval does not contain 0 (Section 6.2).
Before, after	The test time steps 41–42 and 43–49, on the two sides of the dark-market closure.

Table 2: The six families. "Kept" is the number of columns after the redundancy filter of Section 4.4.

Family	Source	All	Kept	Question it answers
LF	Elliptic, opaque	93	93	Local features of the transaction (anonymous)
AF	Elliptic, opaque	72	72	Summaries of the neighbours (anonymous)
A	computed by us	8	6	How many links does the transaction have?
B	computed by us	12	11	What position does it hold in the money flow?
C	computed by us	7	7	What shape does its neighbourhood have?
D	Elliptic++	14	9	What are its economic data?

Table 3: The seven menus and why each one exists.

Menu	Families	Columns	Why we run it
`local`	LF	93	Reference: the local features used in the literature, without the time step.
`local_agg`	LF, AF	165	Reference: the complete original feature set, which is the result to beat.
`graph`	A, B, C	24	What does structure alone give? This is the experiment suggested by the project proposal.
`econ`	D	9	What do a few named economic columns give?
`transparent`	A, B, C, D	33	The main question: how good is a model that uses only features we can explain?
`local_agg_graph`	LF, AF, A, B, C	189	Does structure add information to the original features? D is left out because it repeats LF.
`none`	no column	0	Every node gets the input $x = 1$, so a GNN sees only the edges. This measures what message passing gets from the graph alone. It also gives a lower reference that uses no hand-made feature.

3 Data

We use only the transaction part of Elliptic++ [2], which contains the original Elliptic transactions with the same labels and edges, plus 17 columns. The dataset has 203,769 transactions and 234,355 edges over 49 time steps. Of these transactions, 4,545 are illicit, 42,019 are licit and 157,205 (77%) are unknown (Figure 20 in the appendix). The time step is used only to split the data. It is never a feature, because a model could memorize it and it would say nothing about the future.

The split follows time. We train on time steps 1–30, validate on 31–40 and test on 41–49, as shown in Table 4. We never shuffle, because a real detector always predicts the future. There is no edge between time steps, so nothing leaks from one split to another. Validation is kept separate because tuning and early stopping need it, and the test set is read only once, in the final runs.

Table 4: The three splits. Evaluation rows are labelled transactions with the Elliptic++ data.

Split	Time steps	Transactions	Evaluation rows	Illicit	Prevalence
Train	1–30	123,287	26,750	2,954	0.110
Validation	31–40	38,316	9,429	1,067	0.113
Test	41–49	42,166	9,866	524	0.053

The closure of a dark market falls inside the test set. Figure 1 shows the share of illicit transactions per time step. It falls from 11.1% at time step 42 to 1.75% at time step 43. Weber et al. [1] attribute this to the closure of a large dark market and report that their models stop finding illicit transactions at that point. The fall is what we measure. The cause is their reading, and our data alone cannot prove it. This event also halves the prevalence of the test set and it turns out to be central to our results (Section 7).

The graph is almost a forest. Every time step is one connected component of 1,089 to 7,880 transactions. The graph is directed and has no cycles, because a transaction can only spend outputs that already exist. Only 3.5% of the nodes are in a triangle, and there are barely more edges than nodes (Figure 21 in the appendix). Figure 2b shows a small piece of it.

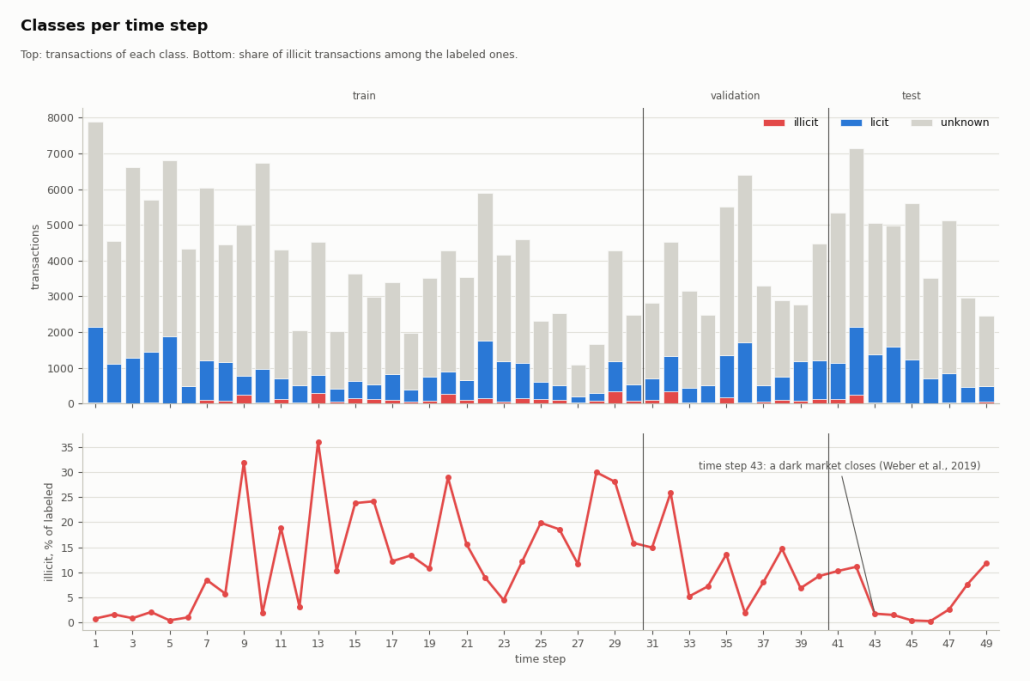
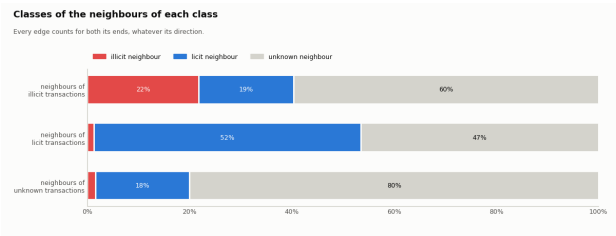
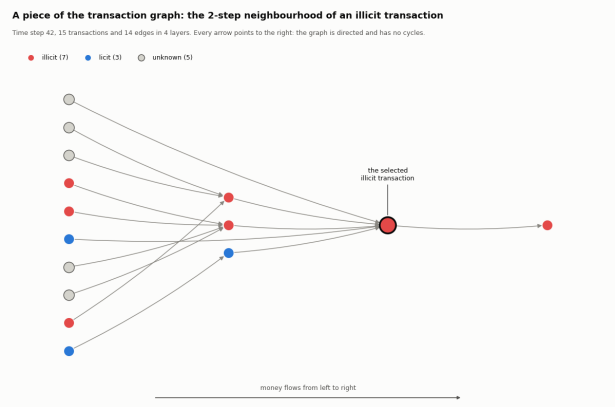


Figure 1: Classes per time step. Top: transactions of each class. Bottom: share of illicit transactions among the labelled ones. The annotation at time step 43 follows Weber et al. [1].

Illicit transactions have illicit neighbours. Figure 2a shows that 22% of the neighbours of illicit transactions are illicit, against about 1% for licit transactions. Among labelled neighbours, the share is 54%. This is the reason to try graph neural networks at all: a transaction may take on the colour of its neighbourhood.



(a) Classes of the neighbours of each class.



(b) The 2-step neighbourhood of one illicit transaction.

Figure 2: Who is linked to whom. Neighbours are often unknown, and illicit transactions often sit close to each other.

Unknown transactions stay in the graph. They have no label, so they stay out of the loss and the metrics, but they stay in the graph as structure and as input from the neighbours. Removing them would leave 15.6% of the edges, and only 54.7% of the illicit transactions would still have a neighbour, which would remove most of the signal a graph model can use.

Every menu is measured on the same rows. 965 transactions have no Elliptic++ data, and none of them is illicit. The 519 labelled ones among them are left out of the loss and the metrics, so

that all menus are compared on the same transactions. Their missing columns take the median of the train, because they still feed their neighbours. This gives the test prevalence of 0.053.

4 Features

4.1 Opaque features: LF and AF

Elliptic has 93 local features (LF, without the time step) and 72 aggregate features (AF), which summarize the neighbours of a transaction. Their meaning is not published. We keep them as the reference to beat.

4.2 Economic features: family D

Elliptic++ adds 17 named columns. Their number goes down in two stages:

- 3 are exact copies of other columns and are removed: `total_BTC` equals `out_BTC_total`, while `in_txs_degree` and `out_txs_degree` equal the degrees of the graph. This leaves 14.
- 5 of those 14 are near-copies of `in_BTC_total` and are removed by the filter of Section 4.4, because most transactions have one input address and for them totals, maxima and means coincide. This leaves the **9 columns of family D that the models actually use**.

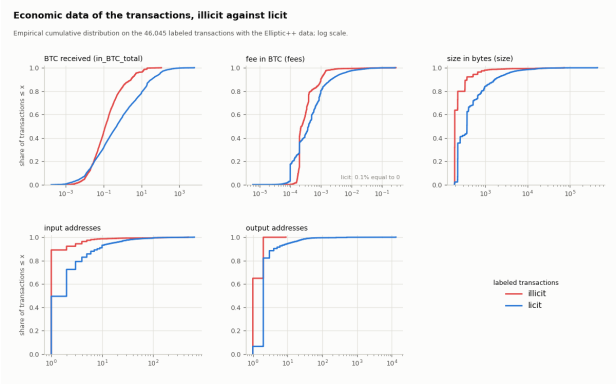
Table 5: Family D. All values are non-negative. The last five rows are removed by the filter.

Column	Meaning	High value means
`in_BTC_total`	BTC received by the transaction	it moves a large amount
`fees`	fee in BTC, equal to total input minus total output	the sender paid a high fee
`size`	size of the transaction in bytes	many inputs or outputs
`num_input_addresses`	number of input addresses	it merges money from many addresses
`num_output_addresses`	number of output addresses	it splits money towards many addresses
`in_BTC_min`, `in_BTC_median`	smallest and median input amount	even the smaller inputs are large
`out_BTC_min`, `out_BTC_median`	smallest and median output amount	even the smaller outputs are large
`out_BTC_total`	BTC sent; near-copy of `in_BTC_total`	removed
`in_BTC_max`, `in_BTC_mean`	largest and mean input amount	removed
`out_BTC_max`, `out_BTC_mean`	largest and mean output amount	removed

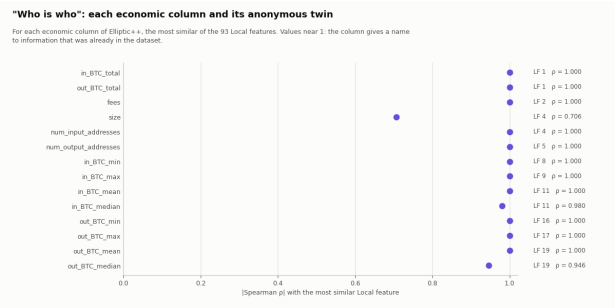
Figure 3a compares illicit and licit transactions on five columns. We drew these five by hand, one per kind of information (amount, fee, size, inputs, outputs), because the other columns are mostly variants of the amount and 17 nearly equal panels would say less. A curve that is higher and further left means smaller values. Illicit transactions are clearly smaller and simpler: about 90% have a single input address, against about 50% of the licit ones.

Most economic columns are duplicates of local features. In Figure 3b, 11 of the 14 columns have $|\rho| \approx 1$ with one LF, the two medians have 0.98 and 0.95, and only `size` stands apart with 0.71. Spearman 1 means the same order of the transactions; the numeric values can differ. So family D gives a name to information that was already in the dataset.

All the opaque columns stayed. We handled the duplicates through the menus: **D and LF are never in the same menu**, as Table 3 shows. Six of the 9 columns of D still have the same order as



(a) Cumulative distribution of five columns, illicit against licit.



(b) For each economic column, the most similar LF.

Figure 3: The economic columns separate the classes (a), and most of them repeat a local feature (b). All Spearman values are on the 202,804 transactions with the Elliptic++ data.

an LF, and this does no harm, because the menus that contain D contain no LF. It also means that "D added to LF" is not a useful experiment, since we would expect a gain close to 0. The comparison we run instead is `transparent` − `local`, a swap of 93 anonymous columns for 33 named ones.

4.3 Graph features: families A, B and C

We compute 27 features from the edges alone. The list starts from the examples in the project proposal [**?**]: degrees, centralities, clustering and graphlets. We keep degrees and centralities and we change the rest, because of what the data look like:

- Clustering and triangle counts are 0 for 96.5% of the nodes, so they carry almost no information. Family C measures shapes that exist in a near-tree instead: fans, chains and diamonds.
- Cycles cannot exist in this graph, so we do not count them.
- Ollivier–Ricci curvature is included because it works without triangles. We prefer it to Forman curvature, which on this graph is almost a function of the degrees.

Common rules. Every feature is computed inside its own time step, which is exact because no edge crosses time steps. No feature uses labels. A value is divided by the size of the time step only when the raw value sums over all nodes or all pairs (PageRank, HITS, betweenness, closeness), because then it grows with the graph by construction. Counts of what a node touches or reaches stay raw: a chain of 10 steps is 10 steps long in any time step. Every formula was first tested on a small graph with values computed by hand.

Notation. $G_t = (V_t, E_t)$ is the directed graph of time step t, with n_t nodes. $N^-(v)$ and $N^+(v)$ are the predecessors and successors of v, $d^-(v)$ and $d^+(v)$ their numbers. $\vec{d}(u, v)$ is the length of the shortest directed path. $\hat{G}_t$ is the undirected graph, with distance dist, neighbours $N(v)$ and degree $\deg(v) = d^-(v) + d^+(v)$. A is the adjacency matrix and $\hat{A} = A + A^\top$. A $\times$ after a name marks a column removed by the filter of Section 4.4.

Table 6: Family A: how many links does the transaction have?

Feature and domain	Definition	Meaning
`in_deg` $\mathbb{N}$	$d^-(v)$	Earlier transactions whose outputs v spends. High: v merges money from many transactions. Zero: no input comes from a transaction in the graph.
`out_deg` $\mathbb{N}$	$d^+(v)$	Later transactions that spend outputs of v. High: the money goes to many transactions. Zero: nothing is spent inside the time step.
`anc2` $\{0, \dots, n_t - 1\}$	$\lvert\{u : \vec{d}(u, v) = 2\}\rvert$	Transactions exactly two steps back. High: a wide history just behind v.
`desc2` $\{0, \dots, n_t - 1\}$	$\lvert\{w : \vec{d}(v, w) = 2\}\rvert$	Transactions exactly two steps ahead. High: the money spreads widely two steps later.
`pred_out_mean` $\{0\} \cup [1, \infty)$	$\frac{1}{d^-(v)} \sum_{u \in N^-(v)} d^+(u)$ (0 if $d^-(v) = 0$)	Into how many pieces did the predecessors split their money? High: v comes from a distribution and has many siblings.
`pred_out_max` × $\mathbb{N}$	$\max_{u \in N^-(v)} d^+(u)$	The same for the most extreme predecessor.
`succ_in_mean` $\{0\} \cup [1, \infty)$	$\frac{1}{d^+(v)} \sum_{w \in N^+(v)} d^-(w)$ (0 if $d^+(v) = 0$)	With how many other transactions is the money of v merged when it is spent? High: v flows into a consolidation.
`succ_in_max` × $\mathbb{N}$	$\max_{w \in N^+(v)} d^-(w)$	The same for the most extreme successor.

Table 7: Family B: what position does the transaction hold in the money flow of its time step?

Feature and domain	Definition	Meaning
`pr_fwd` $[0.15, n_t)$, mean 1	PageRank on G_t with $\alpha = 0.85$, times n_t: $PR(v) = \frac{1-\alpha}{n_t} + \alpha \sum_{u \in N^-(v)} \frac{PR(u)}{d^+(u)} +$ $\frac{\alpha}{n_t} \sum_{u:\, d^+(u)=0} PR(u)$	How much money accumulates on v from upstream. High: v receives from many transactions that are themselves well fed. 1 is the average of the time step.
`pr_rev` same domain	The same formula with reversed edges	High: v feeds much of what lies downstream.
`betw` $[0, 1]$	Betweenness on $\hat{G}_t$: $\binom{n_t-1}{2}^{-1} \sum_{\{s,z\} \not\ni v} \frac{\sigma_{sz}(v)}{\sigma_{sz}}$, where σ_{sz} counts shortest paths	How much v is a bridge. High: many shortest connections between other nodes pass through v. Zero: a leaf of the graph.
`harm_in` $[0, n_t - 1]$	$\sum_{u \neq v,\ \vec{d}(u,v) < \infty} \frac{1}{\vec{d}(u,v)}$	All the history behind v, with closer ancestors weighing more. High: a long and wide history. Zero: no ancestors. It is left raw because only ancestors count, so it does not grow with the time step.
`harm_out` $[0, n_t - 1]$	$\sum_{w \neq v,\ \vec{d}(v,w) < \infty} \frac{1}{\vec{d}(v,w)}$	All the future in front of v. High: the money reaches many later transactions.
`eig` $[0, 1]$	$\hat{A}x = \lambda_{\max} x$, $x > 0$, $\mathrm{eig}(v) = x_v / \max_u x_u$; values below 10^{-12} are set to 0	Is v linked to nodes that are themselves well linked? High: v is near the most connected area. The undirected graph is used because the directed version does not exist on a graph without cycles.
`hub` $[0, n_t)$, mean 1	HITS: $h \propto Aa$, $a \propto A^\top h$, $\sum_v h_v = 1$; $\mathrm{hub}(v) = n_t h_v$	High: v sends money to transactions that receive from many.
`auth` $[0, n_t)$, mean 1	HITS: $\sum_v a_v = 1$; $\mathrm{auth}(v) = n_t a_v$	High: v receives money from transactions that send to many.

Feature and domain	Definition	Meaning
`core` $\{1, 2, \dots\}$	$\max\{k : v \in k\text{-core}(\hat{G}_t)\}$	How deep v sits in a dense group with redundant links. High: a dense area. 1: tree-like periphery.
`katz_in` $\times$ $[0, \infty)$	$\sum_{k\geq 1} \gamma^k [(A^\top)^k \mathbf{1}]_v$, $\gamma = 0.5$	How many money paths end in v, each extra step weighing half. High: many paths converge on v.
`katz_out` $[0, \infty)$	$\sum_{k\geq 1} \gamma^k [A^k \mathbf{1}]_v$, $\gamma = 0.5$	How many money paths start from v. High: many paths leave v.
`clo` $(0, 1]$	$\frac{n_t - 1}{\sum_{u \neq v} \text{dist}(u,v)}$ on $\hat{G}_t$	Inverse of the mean distance to all other nodes. High: v is in the centre of the time step. Low: v is in the periphery.

Table 8: Family C: what shape does the neighbourhood of the transaction have?

Feature and domain	Definition	Meaning
`is_fan_in` $\{0, 1\}$	$\mathbf{1}[d^-(v) \geq \tau^-]$, with τ^- the smallest degree that flags at most 5% of the train nodes; here $\tau^- = 3$	1: v collects money from many transactions (consolidation). A plain percentile would land on a tie, since most degrees are 1 or 2.
`is_fan_out` $\{0, 1\}$	$\mathbf{1}[d^+(v) \geq \tau^+]$, in the same way; here $\tau^+ = 3$	1: v splits money towards many transactions (distribution).
`chain_len` $\mathbb{N}$	If $d^-(v) = d^+(v) = 1$: the size of the component of v among such nodes. Otherwise 0	High: v is a link of a long chain with one input and one output, like a peel chain. Zero: v is in no chain.
`diam_out` $\mathbb{N}$	$\lvert\{w : \lvert\{m : (v, m), (m, w) \in E_t\}\rvert \geq 2\}\rvert$	High: the money of v often splits and merges again two steps later (scatter-gather).
`diam_in` $\mathbb{N}$	$\lvert\{u : \lvert\{m : (u, m), (m, v) \in E_t\}\rvert \geq 2\}\rvert$	High: v often merges money that had split two steps before.
`orc_mean` $[-2, 1]$	$\frac{1}{\deg(v)} \sum_{u \in N(v)} \kappa(v, u)$, with $\kappa(x, y) = 1 - W_1(m_x, m_y)$. m_x keeps half of the mass on x and splits the other half among its neighbours; W_1 is the optimal transport distance, solved exactly	Ollivier–Ricci curvature. High ($\kappa > 0$): the neighbourhood of v overlaps with those of its neighbours, as in a dense or star-like area. Low ($\kappa < 0$): they move apart, as in a tree or a bottleneck.
`orc_min` $[-2, 1]$	$\min_{u \in N(v)} \kappa(v, u)$	The most bottleneck-like link of v. Low: at least one link of v is a bottleneck.

The graph features are new information. Their highest $|\rho|$ with any of the 165 opaque columns is 0.37 for family A, 0.56 for B and 0.43 for C (Figure 22b in the appendix).

4.4 The redundancy filter

Near-copies make importance hard to read, because correlated columns share their credit. So we remove them, with this rule. The 41 transparent columns are visited in the order A, B, C, D. A column is removed if a column already kept has

$$\min\left(|\rho_{\text{all}}|,\ |\rho_{\text{illicit}}|,\ |\rho_{\text{licit}}|\right) > 0.95,$$

where ρ is the Spearman correlation on the train time steps: on all nodes, on the illicit evaluation rows and on the licit ones. We use Spearman because the distributions have long tails and only the order of the values matters to a tree. We check three groups because illicit transactions are only 2.4% of the train nodes: two columns could look like copies overall and differ exactly on the illicit ones. Only train labels are used, so nothing leaks.

The filter removes 8 columns: `pred_out_max` (kept: `pred_out_mean`), `succ_in_max` (kept: `succ_in_mean`), `katz_in` (kept: `harm_in`), and `out_BTC_total`, `in_BTC_max`, `in_BTC_mean`, `out_BTC_max`, `out_BTC_mean` (kept: `in_BTC_total`). This leaves 33 transparent columns: 24 from the graph and 9 economic.

How to read the correlation plots. Figure 4 is the decision. The × is next to the *name* of a removed column on the axes, and it never appears inside the matrix. A black outline marks the pair that caused a removal: the removed column and the column kept in its place. The matrix is symmetric, so 8 removals give 16 outlined squares. These squares are only part of the correlations above 0.95. The 41 columns have 19 pairs above 0.95 on all nodes and 16 pairs above 0.95 in all three groups, and only 8 of these are outlined, for two reasons:

- The rule checks three groups, while the colour shows only "all nodes". For example `in_BTC_mean` and `in_BTC_median` have 0.977 on all nodes and 0.996 on the illicit rows, but 0.926 on the licit rows. So `in_BTC_median` stays, and its very red square has no outline.
- Each removed column has one partner. `out_BTC_total` and `in_BTC_max` have 0.991 and no outline, because both were removed as copies of `in_BTC_total`.

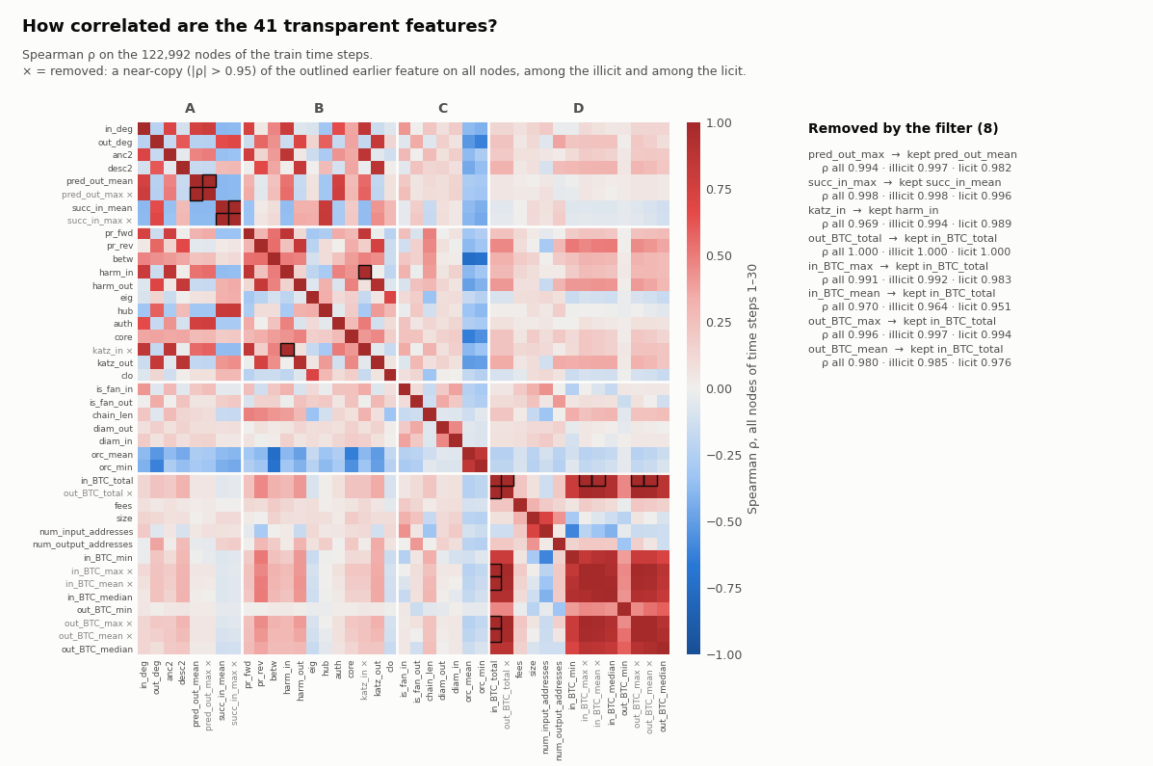
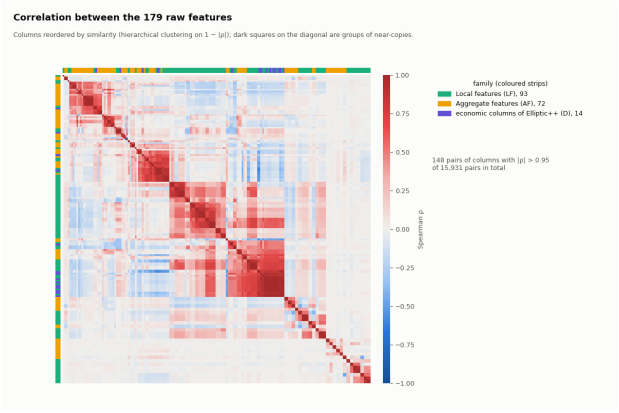
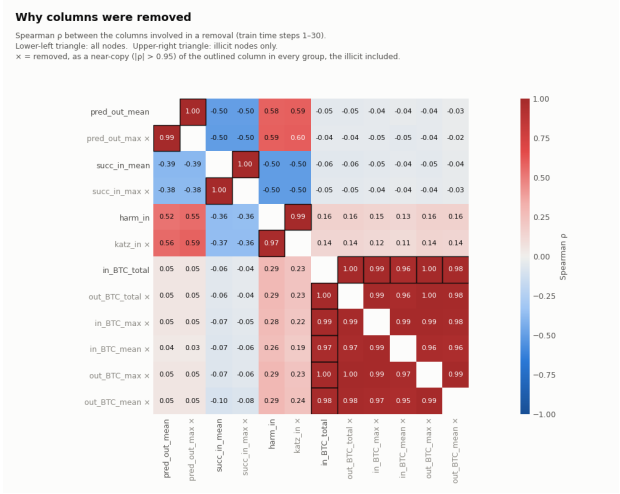


Figure 4: Spearman correlation of the 41 transparent features on the train time steps, with the 8 removals listed on the right.

Three plots, three zoom levels. Figure 5a is the overview of the raw dataset: 179 columns (LF, AF and D) without names, on all time steps, reordered by similarity. It has no A, B or C, because they are computed later. It shows that the dataset is redundant, with 148 pairs above 0.95. Figure 4 is the decision, on the 41 named columns and only on the train time steps, because a decision must not see validation and test. Figure 5b is the proof: only the 12 columns involved in a removal, with the values written, all nodes below the diagonal and illicit rows above it. It shows that the copies hold on the illicit transactions too, so removing them is safe.



(a) The 179 raw columns, on all time steps. Dark squares on the diagonal are groups of near-copies.



(b) Why the columns were removed: all nodes below the diagonal, illicit rows above it.

Figure 5: The two other views of the correlations.

4.5 The menus

A menu is a question: each comparison between two menus answers one precise point. Table 3 in Section 2 gives the seven menus and the reason for each one. We use the same search space, the same number of trials and the same metric on every menu with columns, so that the differences come from the columns.

Transformations. Trees use raw values, because only the order of the values matters to them. Logistic Regression, MLP and GNNs need comparable scales, and some columns have huge tails (`katz_in` reaches about $7 \cdot 10^{10}$). For them each column gets a $\log(1+x)$, or its signed version, when its skewness is above 2, and then a z-score, with the parameters estimated on the train time steps.

5 Models and training

Table 9: The models. h_v is the hidden vector of node v and $N(v)$ its neighbours. The edges are made undirected for the GNNs, so that a message can come both from where the money was and from where it goes.

Model	Rule	Why it is here	Trials
Logistic Regression	a linear score of the columns	linear reference: what a straight line can do	20
Random Forest	500 trees	the best model in [1]	20
XGBoost	up to 2,000 boosted trees	strong on tabular data	40
MLP	dense layers, no edges	sees the same columns as a GNN, so GNN − MLP measures what the graph adds	40
GraphSAGE	$h'_v = W_1 h_v + W_2 \,\mathrm{agg}_{u \in N(v)} h_u$	simple message passing	40
GATv2	$h'_v = \sum_{u \in N(v) \cup \{v\}} \alpha_{vu} W h_u$, with learned attention α_{vu}	among the best in [3]; it can weigh neighbours differently	40
ChebNet	$h' = \sum_{k=0}^{K-1} \theta_k T_k(\tilde{L}) h$, with $K \in \{2,3,4\}$	the best in [3]; $K=1$ would ignore the graph	40 (20 on `none`)
GIN	$h'_v = \mathrm{MLP}\big((1+\varepsilon) h_v + \sum_{u \in N(v)} h_u\big)$	only on `none`: its sum can see degrees	20

Which model on which menu. The first seven models run on the six menus with columns. The menu `none` runs only ChebNet and GIN. With a constant input, GATv2 and GraphSAGE with mean aggregation give the same output on every node, so they cannot learn anything there. GIN sums, and the normalized Laplacian of ChebNet depends on the degrees, so these two can see the structure. The models without edges have no input at all on this menu, so their reference is the random classifier. The GCN is left out because it was the weakest model in [3]. We also use only the transaction graph, without the wallets of Elliptic++ used in [4], because our question is about the features of a transaction.

Tuning. We tune each pair once, with Optuna: the first 10 trials are random and the next ones follow the good ones (LogReg explores a full grid). The objective is the PR-AUC on validation, because the classes are unbalanced and this metric does not need a threshold. The search spaces are in Appendix B. Trials of neural models that are clearly worse than earlier ones are stopped early, to save time. The budget was set after a timing test, for a total of 1,480 trials. The number of trials is the same for every menu of a model, so that no menu gets a better search.

Variants of the GNNs. GraphSAGE, GATv2 and ChebNet can use the three variants of [3]: base, with a final linear layer, or with a linear layer and a skip connection. We treat the variant as a hyperparameter, so that each menu gets the one that fits it.

Final runs. Each pair is trained again from scratch with seeds 0, 1 and 2 and with the hyperparameters of the tuning, which used seed 42. Three seeds let us separate a real effect from a lucky training. The test set is read for the first time here. Neural models use full-batch Adam and binary cross-entropy with a positive weight of 8.06 (licit over illicit rows of the train), because the classes are unbalanced. GNNs train on the graph of time steps 1–30, unknown nodes included, and are scored on the graphs of later periods. Training stops after 50 (MLP) or 100 (GNN) epochs without improvement on validation, with a cap of 1,000 epochs that no best trial reached.

Metrics. The main metric is the PR-AUC on the test set. We also report the ROC-AUC, and the illicit F1 at the threshold with the best F1 on validation. The threshold is applied unchanged to the test set, because choosing it on the test set would make the F1 optimistic. When one model must represent a menu, we take the one with the best validation PR-AUC. We never choose on the test set, because that would make the reported number optimistic.

Reproducibility. We set the seeds of every library. Training the first triple of each model twice with the same seed gave the same scores on all models, with differences below 10^{-15}. All numbers in this report come from saved files, and the notebooks stop if a check fails (Appendix C).

6 Results

6.1 Every model on every menu

Figure 6 shows all the results and Table 10 gives one model per menu (Figure 23 in the appendix has the exact value of every cell).

Trees win when local features are available. On the three menus with LF, XGBoost and Random Forest reach 0.63–0.65, and their intervals do not overlap with those of any other model, whose best is 0.52. These tree models use no edges, so message passing does not make up for the difference. This agrees with [1], where the Random Forest beats the GCNs. Why trees do better

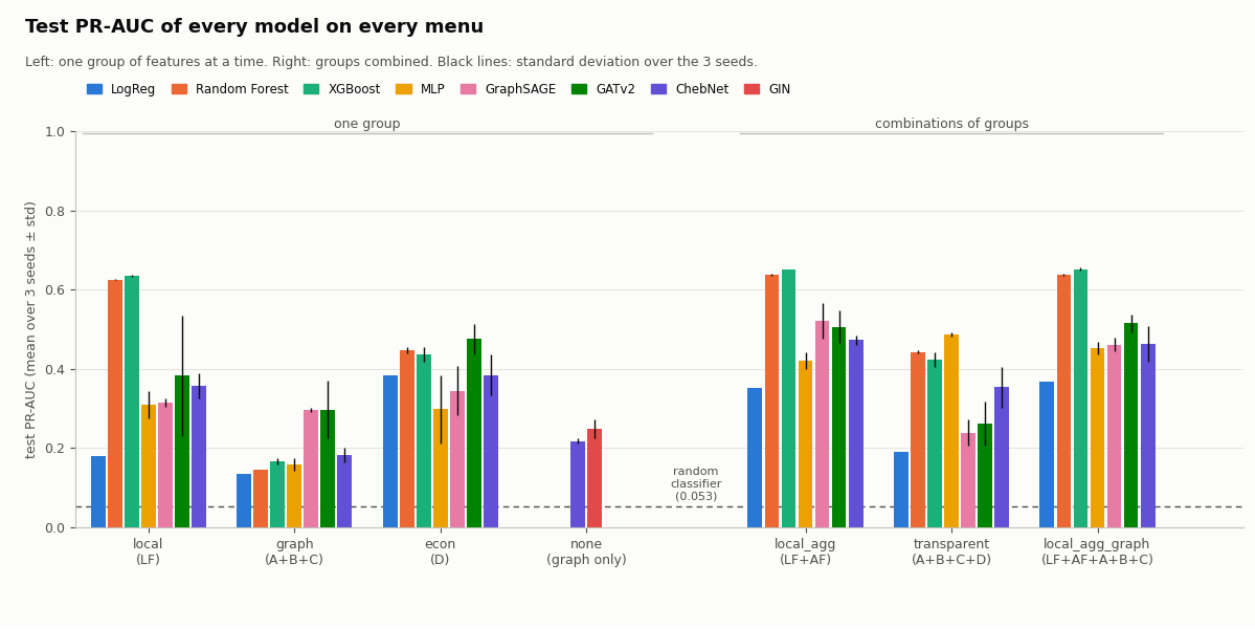


Figure 6: Test PR-AUC of every model on every menu, mean over 3 seeds. Black lines: standard deviation over the seeds. Dashed line: the random classifier.

Table 10: One model per menu, chosen on validation. PR-AUC: mean ± standard deviation over the seeds, with the 95% interval of Section 6.2. Before and after: PR-AUC on the two sides of the closure (Section 7).

Menu	Model	Test PR-AUC	95% interval	F1 illicit	Before	After
`local`	XGBoost	0.635 ± 0.002	0.598–0.671	0.663	0.905	0.037
`local_agg`	XGBoost	0.651 ± 0.001	0.614–0.688	0.702	0.914	0.041
`local_agg_graph`	XGBoost	0.652 ± 0.004	0.613–0.688	0.705	0.914	0.042
`transparent`	XGBoost	0.422 ± 0.018	0.375–0.478	0.568	0.789	0.022
`econ`	XGBoost	0.437 ± 0.018	0.390–0.488	0.567	0.777	0.023
`graph`	GraphSAGE	0.296 ± 0.005	0.258–0.336	0.290	0.525	0.045
`none`	ChebNet	0.218 ± 0.006	0.182–0.253	0.197	0.490	0.025

here is a hypothesis: they use sharp thresholds on tabular columns, as we see for `size` in Section 8.3. The plots show that this happens, and they do not prove why.

What the first results tell us.

1. **The strong signal is in the columns of the single transaction.** A few columns are enough: the 9 columns of D give XGBoost 69% of the PR-AUC of the 93 LF.
2. **Structure alone is weak but real.** On `graph` the PR-AUC goes from 0.134 to 0.297, which is 2.5 to 5.6 times the random classifier. With no column at all, GIN and ChebNet still get 0.248 and 0.218. Compared with the 0.65 of the best menu, these are poor results, which fits the remark in [3].
3. **On structure alone the order flips.** Trees become among the weakest (0.145–0.167) and GraphSAGE and GATv2 the best (about 0.30). GIN with no column (0.248, interval 0.207–0.291) beats XGBoost with 24 structural columns (0.167, interval 0.142–0.197). This suggests that message passing extracts something from the edges that our hand-made structural columns do not fully capture.

Published F1 values on this data come from different splits and different sets of rows, so we do not compare our numbers with them directly.

6.2 What each group adds

Each panel of Figure 7a answers one question by comparing two menus for the same model. Figure 7b compares a GNN with the MLP on the same menu, which measures what the edges add.

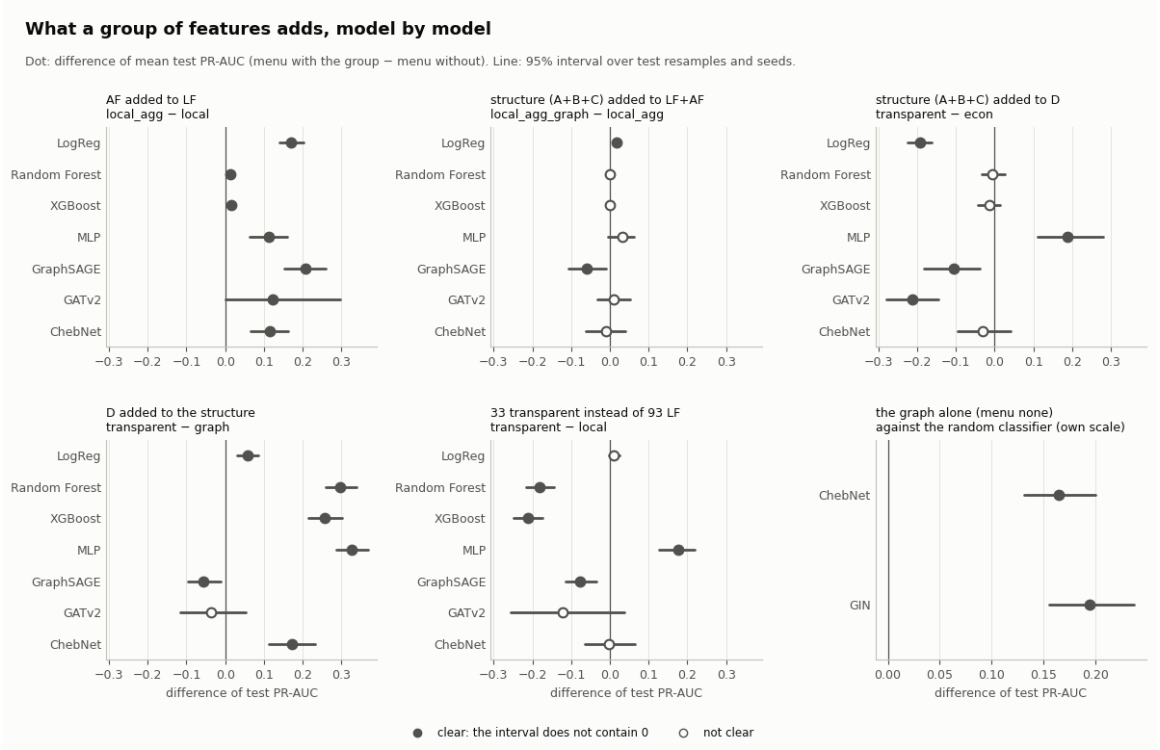
What the line means. The line answers this question: with another similar test set and other training seeds, how much would this difference change? It counts both sources of noise, the transactions in the test and the seed. We compute it with a two-level bootstrap. Take GATv2 in the panel "AF added to LF":

1. Each side has 3 models, one per seed. Their test PR-AUC is 0.541, 0.462, 0.515 on `local_agg` and 0.425, 0.214, 0.510 on `local`. The dot is the difference of the means: $0.506 - 0.383 = +0.123$.
2. From the 9,866 test rows we draw 1,000 alternative test sets of the same size, with replacement, so some rows appear twice and others never. They are the same 1,000 sets for every model. We compute the PR-AUC of every model on each of them.
3. On each alternative test set we draw 3 seeds at random with replacement for each side, average them and subtract. Table 11 shows two possible draws on the first alternative test set.
4. This gives 1,000 differences. We sort them and remove the lowest 2.5% and the highest 2.5%. What remains is the line: from 0.0003 to 0.296.

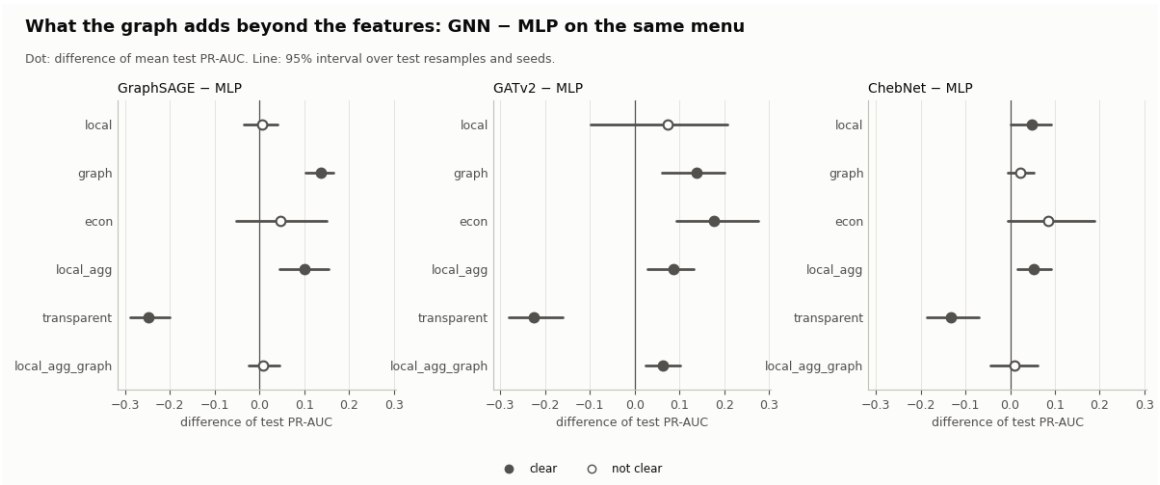
Table 11: Two draws on one alternative test set, where the three seeds score 0.5805, 0.4953, 0.5580 on `local_agg` and 0.4755, 0.2296, 0.5271 on `local`.

Seeds drawn for `local_agg`	Seeds drawn for `local`	Difference
0, 2, 2 (mean 0.5655)	1, 1, 0 (mean 0.3116)	+0.254
1, 1, 0 (mean 0.5237)	2, 2, 2 (mean 0.5271)	−0.003

This example has a long line because seed 1 went much worse than the others on `local`: here the noise comes from the seeds. XGBoost in the same panel has a very short line (+0.017, from 0.008 to 0.027), because its seeds give almost the same model and because both sides use the same alternative



(a) What a group of features adds. Dot: difference of mean test PR-AUC. Line: 95% interval.



(b) What the graph adds: GNN minus MLP on the same menu.

Figure 7: Differences between menus and between models. Filled dot: the interval does not contain 0 (a clear difference).

test sets, so a test set that is easy for one is easy for the other. A single XGBoost model has an interval about 0.07 wide, while the difference has one about 0.02 wide.

A filled dot means that the sign of the difference is clear. It does not mean that the difference is large: Random Forest has +0.013 and a filled dot. A hollow dot means that these data cannot tell the sign, which is different from saying that there is no effect.

What the differences say.

- **AF added to LF** helps every model. The gain is large for LogReg and the networks (+0.11 to +0.21) and small for the trees (+0.013, +0.017).
- **Structure added to LF + AF** gives a clear gain only to LogReg (+0.017). GraphSAGE clearly loses (−0.059), and the other five models show no clear change. Yet these columns are new (Section 4.4): they are new as correlations and they add no predictive power. A possible reason is that AF already summarizes the neighbours.
- **D added to structure** gives a large gain to most models (Random Forest +0.30, XGBoost +0.26, MLP +0.33).
- **Structure added to D** does nothing clear for the trees, helps the MLP (+0.19) and hurts GATv2 (−0.21), LogReg (−0.19) and GraphSAGE (−0.11).
- **33 transparent columns in place of 93 LF** cost the trees about 0.2, while the MLP gains +0.18.
- **The graph alone against chance** is clearly above it for both models.
- **GNN against MLP:** the graph helps on `graph` (GraphSAGE and GATv2) and on `local_agg` (all three), and it clearly hurts on `transparent` (all three, from −0.13 to −0.25).

Limits of the interval. There are only 3 seeds. The alternative test sets treat transactions as independent, while they sit in a graph. They resample the same period, so the interval does not cover a future change in the data. And the 55 comparisons have no correction for multiple tests, so a borderline case like the GATv2 example above could be chance.

6.3 Training curves

Figure 8 shows the training of two models on two menus, on the left, and all the models on the right.

The dashed line is early stopping. The curves continue past the line because training stops only after a number of epochs without improvement, and those extra epochs are thrown away. The MLP on `local` is best at epoch 53 and stops at 103, and in between its validation PR-AUC falls from 0.892 to 0.782.

Early stopping picks the point where validation is highest. Overfitting can already be there at that point: GraphSAGE on `local_agg_graph` has train about 0.99 and validation 0.805 at epoch 82.

How the two sides fit together. The left side is the film of four cases and the right side is the last frame of all cases. The vertical distance between the two curves on the dashed line is about the value in the heatmap: 0.04 and 0.17 for the MLP, 0.08 and 0.16 for GraphSAGE.

A gap is more than overfitting. The split follows time, so the gap also contains the change of the data over time. Logistic Regression on `local` has a gap of 0.41 (train 0.93, validation 0.52), and a model with 93 coefficients on almost 27,000 rows cannot memorize that much. The column `graph`

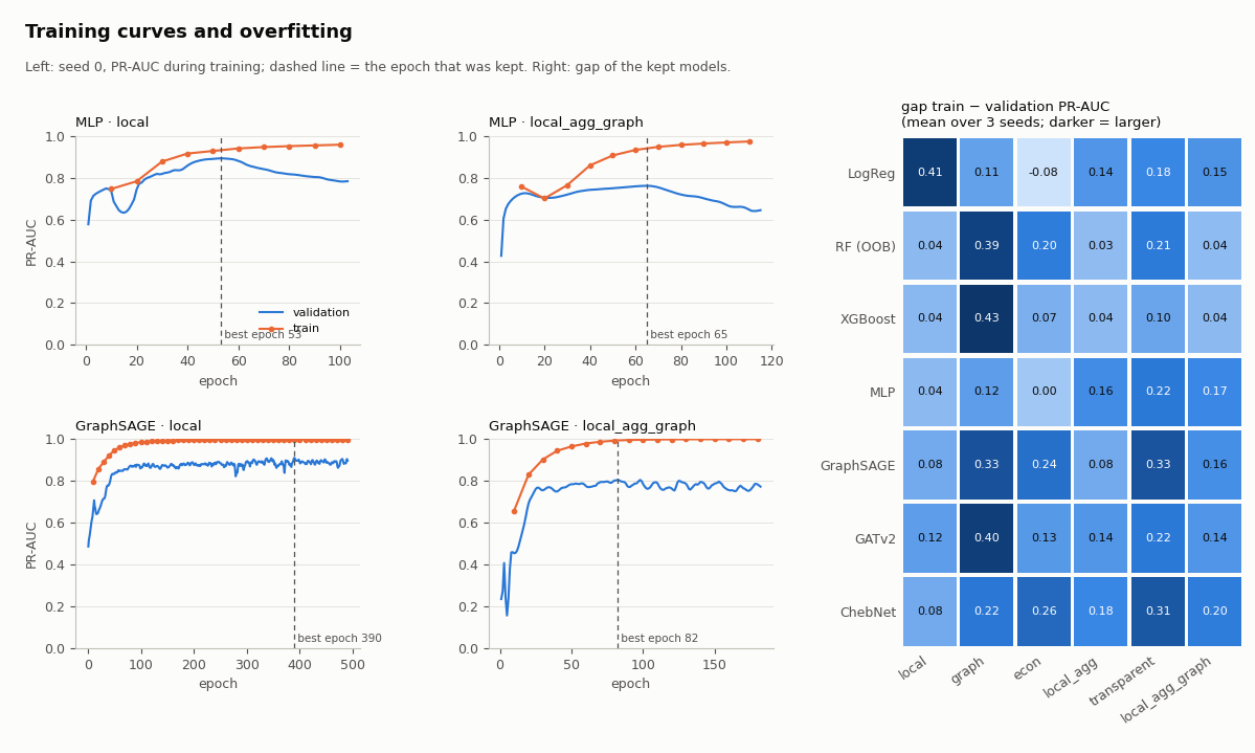


Figure 8: Left: seed 0, PR-AUC during training; the dashed line is the epoch with the best validation PR-AUC, which is the model we keep. Right: PR-AUC of train minus PR-AUC of validation for the kept models, mean over 3 seeds. For Random Forest the train value is the out-of-bag score, because on its own rows it is almost always 1.

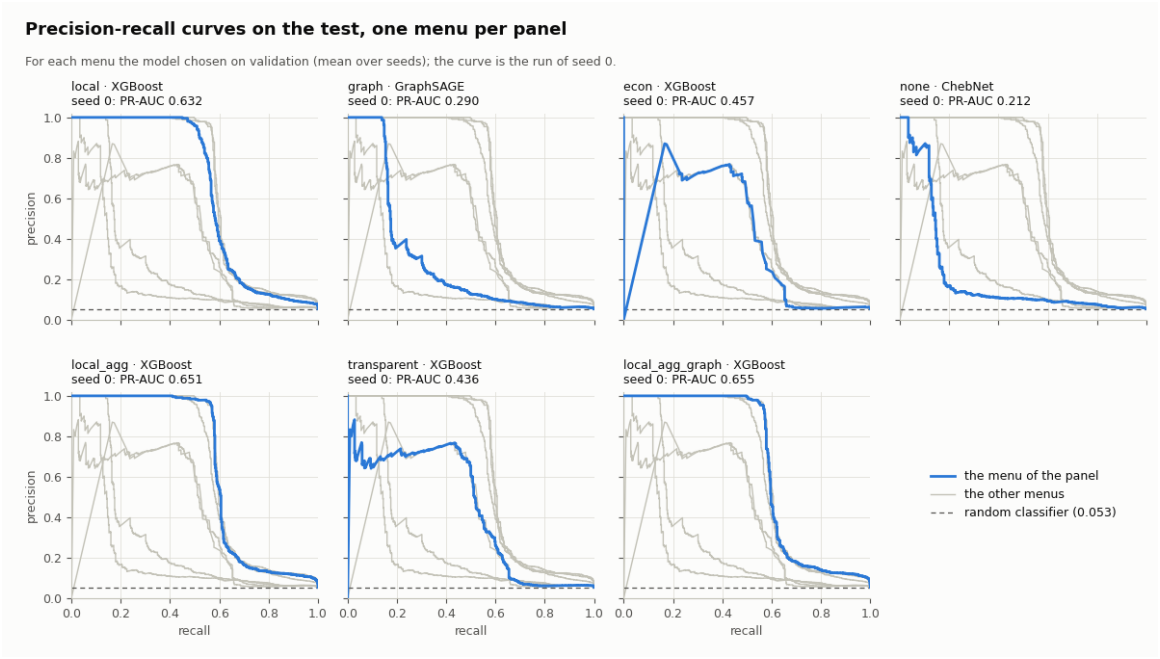
is dark for most models, which suggests that structural patterns transfer badly from one period to the next.

From validation to test. The PR-AUC falls from validation to test in all 132 triples and the ROC-AUC in 128. XGBoost on `local_agg` goes from 0.961 to 0.651 in PR-AUC and from 0.988 to 0.884 in ROC-AUC. Since the ROC-AUC does not depend on the prevalence, the halved prevalence cannot be the whole story. Section 7 gives the rest.

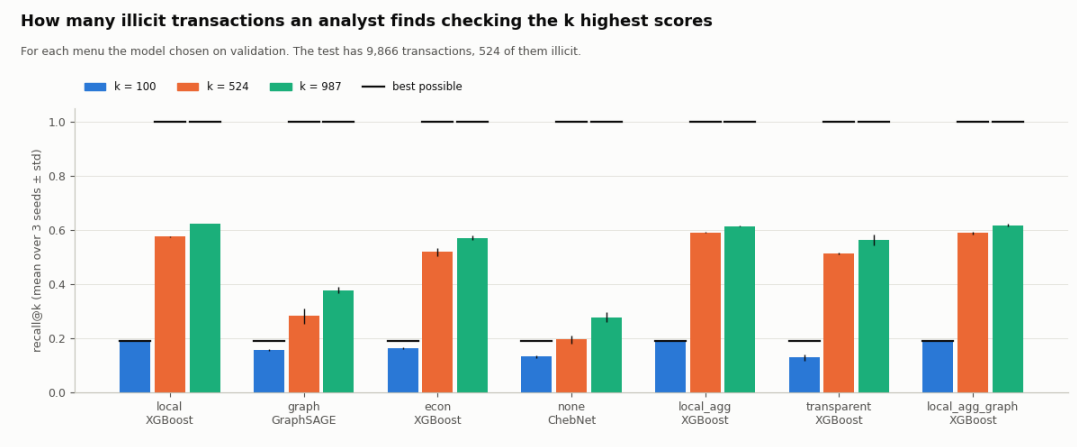
6.4 Precision and recall on the test

Imagine the transactions sorted by score, and walk down the list. Moving right on a curve in Figure 9a means checking more transactions. Recall is the share of illicit transactions found so far, and precision is the share of flagged transactions that are really illicit.

- **A curve that stays high and then collapses** means that the model has a block of illicit transactions it recognizes with certainty, and the others are mixed with the licit ones. XGBoost on `local`, seed 0: the first 210 flagged transactions are all correct (recall 0.40), recall 0.50 still has precision 0.96, and recall 0.60 needs 806 flags at precision 0.39.
- **A curve that falls at once** means that only a few cases are easy, and each further illicit transaction costs many false alarms. GraphSAGE on `graph` has recall 0.10 at precision 1.00, and already 0.32 at recall 0.25. ChebNet on `none` has 0.14 at recall 0.25.
- **A curve that starts low and then rises** (`econ`, `transparent`) means that licit transactions sit at the top of the list. On `econ` the 8 highest scores, all equal, are licit.
- **A tail close to the dashed line** means that the model ranks the illicit transactions left almost at random.



(a) Precision-recall curves. Each panel is one menu, with the model chosen on validation for that menu, seed 0. Blue: this menu. Grey: the other menus.



(b) Recall among the k highest scores, mean over 3 seeds. The black tick is the best possible value.

Figure 9: What an analyst sees when checking transactions from the highest score down.

The top-k view in Figure 9b gives the same message in the language of an analyst. With $k = 100$, XGBoost on the three LF menus reaches recall 0.191, which is the maximum possible, so all 100 are illicit. On `transparent` it reaches 0.128, so about two of every three are illicit. With $k = 524$ the values are 0.59 and 0.51.

The collapse near recall 0.55–0.60 is common to many panels. It has one precise cause, which we study next.

6.5 Which transactions does the graph rescue?

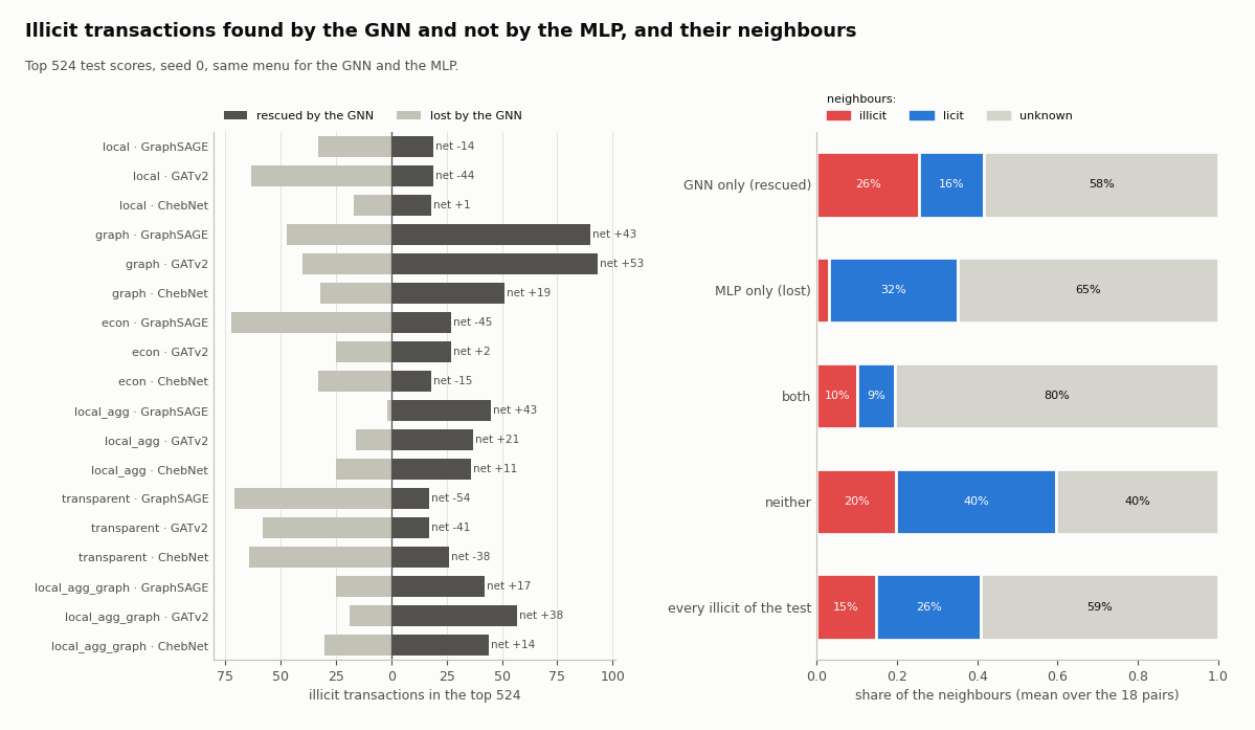


Figure 10: Left: illicit transactions among the 524 highest scores of the GNN and not of the MLP (rescued), and the opposite (lost), seed 0. Right: who the neighbours of these transactions are, mean over the 18 (menu, GNN) pairs. The last row is the reference: all illicit transactions of the test.

Left side. On `graph`, `local_agg` and `local_agg_graph` the net count is positive for all three GNNs. So with the same columns, the edges help a neural network find illicit transactions on these menus. On `transparent` they hurt (net -54, -41, -38).

How this fits with the trees. This comparison is GNN against MLP, and the trees are not in it. Table 12 adds them. On `local_agg` and `local_agg_graph` the order is MLP, then GNN, then trees: the graph gives a neural network part of what trees reach without edges. In these counts the GNN beats the trees only on `graph`, the one menu where structure is all there is. The net count comes from one seed and one cutoff. With 3 seeds and an interval (Figure 7b), the gain is clear for GraphSAGE and GATv2 on `graph`, for all three on `local_agg`, and only for GATv2 on `local_agg_graph`.

Right side. Each row is a group of illicit transactions of the test, according to who places them among its 524 highest scores: the GNN only, the MLP only, both, neither, and last all of them. Each bar splits the neighbours of the group by class, and we compare each row with the last one, which has 15% illicit neighbours. The GNN rescues transactions with many illicit neighbours (26%) and loses those with almost none (3%), so message passing works as expected: a transaction takes on the

Table 12: Illicit transactions among the 524 highest scores, seed 0.

Menu	MLP	GraphSAGE	GATv2	ChebNet	XGBoost	Random Forest
`graph`	115	**158**	**168**	134	116	116
`local_agg`	247	290	268	258	**309**	304
`local_agg_graph`	222	239	260	236	**311**	306
`transparent`	268	214	227	230	268	**270**

colour of its neighbourhood. Transactions found by both have mostly unknown neighbours (80%), and their own columns are enough. Those that neither finds have the highest share of licit neighbours (40%), but also 20% of illicit ones, so illicit neighbours alone do not guarantee detection.

The GNN never sees the labels of the neighbours, only their columns. The class of a neighbour is used by us, afterwards, to read the result. So the plot shows an association.

7 The closure of the dark market

The collapse of the PR curves and the gap between validation and test both called for a closer look at the test set. We added a step that reads the saved scores and retrains nothing. It splits the test set in two periods, fixed in advance from the time step in [1]: **before** (time steps 41–42, 355 illicit, prevalence 10.9%) and **after** (time steps 43–49, 169 illicit, prevalence 2.6%). All models were trained on time steps 1–30, so they never saw the closure.

Because the prevalence changes, the PR-AUC is not fair across the periods, so we also report the ROC-AUC and the *lift* (PR-AUC divided by the prevalence, which is 1 for a random classifier). Intervals are computed inside each period, as in Section 6.2.

Before the closure the models work well, and named features are close to opaque ones. In Figure 11, XGBoost on `local_agg_graph` has a PR-AUC of 0.914 and a ROC-AUC of 0.970 before the closure, close to its validation values (0.961 and 0.987). On `transparent`, XGBoost reaches 0.789 and the MLP 0.813. On `econ`, models reach 0.71–0.81 with only 9 named columns. So the gap between named and opaque features is much smaller than the numbers on the whole test set suggest.

After the closure no feature set works. The same XGBoost falls to a PR-AUC of 0.042 (lift 1.6) and a ROC-AUC of 0.703. Among the 524 highest scores it places 304 illicit transactions from before the closure and 5 from after it. Since the 355 illicit transactions before the closure are 68% of the total, this explains the collapse of the PR curves near recall 0.6 in Section 6.4: at that point the models run out of the illicit transactions they can recognize. Only 28 of the 44 pairs stay clearly above chance on PR-AUC, and none exceeds 0.11.

Prevalence does not explain the fall. Figure 12a shows the recall by time step. It falls at time step 43 on all four menus shown. Time step 49 has a prevalence of 11.8%, about the same as the train, and the recall of XGBoost is 0.006 there. A few points need care: time step 46 has only 2 illicit transactions, so the value 0.5 is one transaction out of two, while GraphSAGE on `graph` finds 64% of the 36 illicit transactions of time step 48, in all three seeds, and stays low elsewhere.

Why: the illicit transactions change shape. Table 13 and Figure 12b compare four groups. Illicit transactions of the train and those before the closure look alike: small, with one input address and often one output address. After the closure they look different. Their median size goes from 192 to 667 bytes, which is above the 372 bytes of licit transactions in the same period. The share under 200 bytes goes from 73% to 0%.

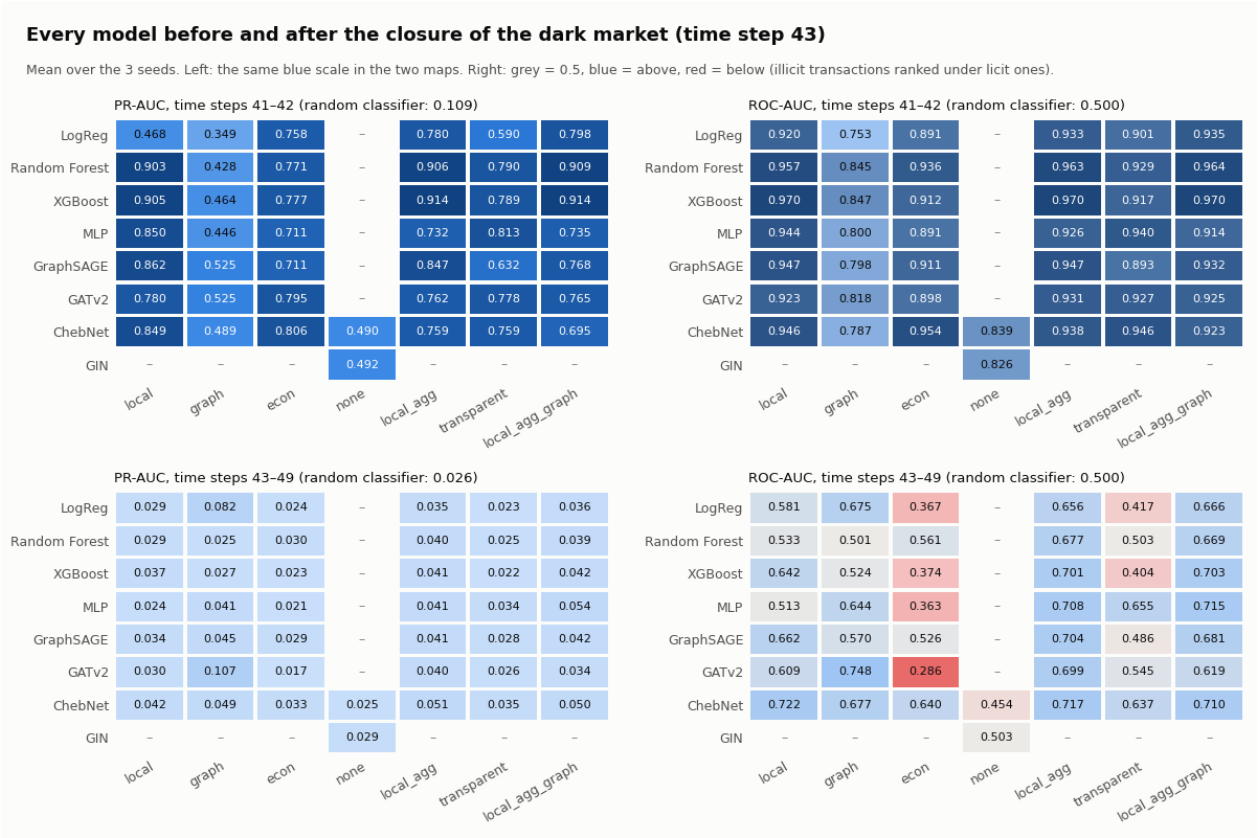
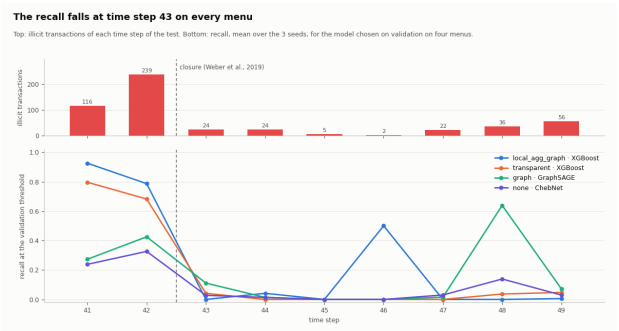
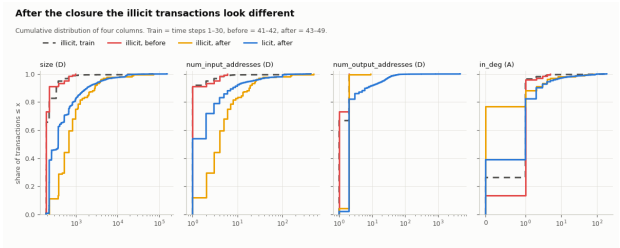


Figure 11: Every model before (top) and after (bottom) the closure, mean over 3 seeds. Left: PR-AUC, with the same blue scale in both maps. Right: ROC-AUC. Grey is 0.5, blue is above it, and red is below it, which means that illicit transactions are ranked under licit ones.



(a) Illicit transactions per time step (top) and recall at the validation threshold (bottom), mean over 3 seeds.



(b) Cumulative distribution of four columns. The yellow curve (illicit, after) moves away from the red and grey ones.

Figure 12: The recall falls at time step 43 (a), when the illicit transactions change shape (b).

Table 13: The profile of the illicit transactions. Train means time steps 1–30, before means 41–42, after means 43–49.

	Illicit, train	Illicit, before	Illicit, after	Licit, after
Share with `size` under 200 bytes	66%	73%	0%	1%
Share with 1 input address	92%	91%	12%	54%
Share with 1 output address	67%	73%	4%	2%
Median `size` (bytes)	192	192	667	372
Median `num_input_addresses`	1	1	4	1
Median `num_output_addresses`	1	1	2	2
Median `in_deg`	1	1	0	1
Median `succ_in_mean`	2	1	18.5	2

Some rules turn against the model. A model that learned "small and simple means illicit" now ranks illicit transactions low. Seven pairs have a ROC-AUC clearly below 0.5 after the closure: LogReg, XGBoost, MLP and GATv2 on `econ` (GATv2 reaches 0.286), LogReg and XGBoost on `transparent`, and ChebNet on `none`. These are the red cells of Figure 11. This is the price of a rule based on a few economic columns: it is easy to read and it is tied to one period.

The graph holds a little better, with low confidence. The highest lift after the closure belongs to GATv2 on `graph` (PR-AUC 0.107, lift 4.2, ROC-AUC 0.748), followed by LogReg on `graph` (0.082, lift 3.2). The family that changes most is D: the mean Kolmogorov–Smirnov distance between illicit transactions of the train and of the after period is 0.59 for D, 0.37 for B, 0.35 for A and 0.14 for C. This fits the idea that structure resists better. We present it as a lead, for two reasons. GATv2 varies a lot with the seed (0.169, 0.107, 0.045, interval 0.049–0.176), and trees on `graph` stay near chance (ROC-AUC about 0.5).

8 Inside the models

Section 6 retrains a model on different menus to see whether a group *improves performance*. This section takes the trained models and asks whether they *use* a group. The two questions are different, so we report both.

8.1 Methods

Permutation importance. We shuffle a column, or a whole family with one common permutation, and measure the drop of the test PR-AUC with no retraining:

$$I_j = \mathrm{AP}(\text{test}) - \frac{1}{R}\sum_{r=1}^{R} \mathrm{AP}(\text{test with } j \text{ shuffled by permutation } r), \qquad R = 5.$$

We use it because it has the same definition for every model. We shuffle inside each time step, so that a drop means "the model uses this column" and cannot come from a value of another period. The permutations are the same for every model.

Attributions. An attribution ϕ_{ij} says how much column j pushes transaction i towards illicit. Permutation has no direction and works on the whole set, so we need this second measure to see a direction and to explain a single transaction. The method depends on the model, because each model allows an exact computation of a different kind:

- Logistic Regression: $\phi_{ij} = w_j(x_{ij} - \bar{x}_j)$, in log-odds.

- Random Forest and XGBoost: TreeSHAP, which is exact on trees, in probability and in log-odds respectively.
- MLP and GNNs: Integrated Gradients in logits, with the mean of the train as baseline x':

$$\phi_{ij} = (x_{ij} - x'_j)\,\frac{1}{m}\sum_{k=1}^{m}\frac{\partial f}{\partial x_j}\Big(x' + \tfrac{k-1/2}{m}(x_i - x')\Big), \qquad m = 50.$$

  For a GNN the gradient is taken on the features of every node of the neighbourhood, so each ϕ_{ij} splits into the part of the transaction itself and the part of its neighbours.

Attributions add up to the output of the model, with a small error for Integrated Gradients. We checked this, together with the reload of every model, in 462 checks that all passed.

Samples and seeds. Importances are means over the 3 seeds. Random Forest and the GNNs are slow to explain, so they use an *explain sample* with all 524 illicit transactions and 1,000 licit ones, with weights to estimate means over the test. Integrated Gradients on the GNNs ran only on seed 0: a timing cell estimated 4.1 hours for three seeds against a budget of 2.5 hours, and it chose one seed by a fixed rule. Plots of a single run use seed 0, fixed in advance. The menu `none` has no column, so it only gets GNNExplainer.

Internal importances. Libraries also give cheap importances (coefficients, impurity, gain). They rank columns in a similar way to SHAP, with a Spearman correlation of 0.83–0.92 for LogReg, 0.75–0.98 for Random Forest and 0.59–0.97 for XGBoost. They use different units, so we do not report them further.

8.2 Which families do the models rely on?

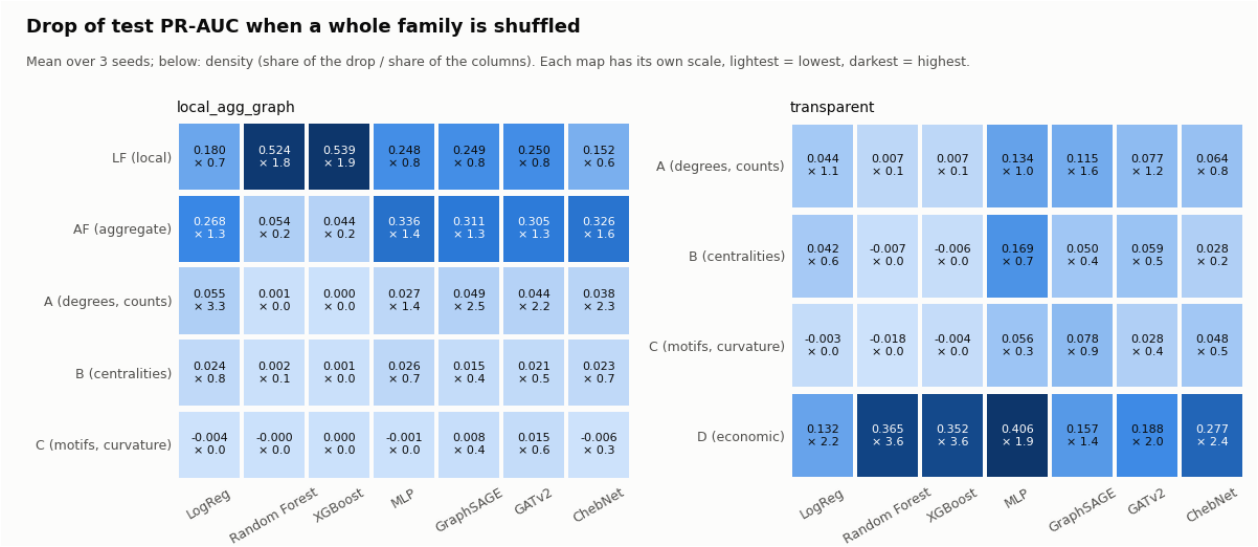


Figure 13: Drop of test PR-AUC when a whole family is shuffled, mean over 3 seeds. Below each value is the density. Each map has its own colour scale.

The value in each cell of Figure 13 answers this question: *if we ruin this family in a trained model, how much PR-AUC does the model lose?*

- **A high value means that the model depends on the family.** LF with XGBoost on `local_agg_graph` is 0.539: with the LF ruined, the PR-AUC goes from 0.652 to about 0.11, and the model is almost blind.

- **A value near 0 means that the model does not need the family.** A, B and C are about 0 for the trees. Those columns may still be useful elsewhere, since `graph` alone is well above chance. This model simply has better columns, or columns that replace them.
- **A slightly negative value is noise.**

The value below it, such as ×1.8, is the **density**: the share of the total drop divided by the share of the columns. For LF with Random Forest, the LF make about 90% of the drop and are 49% of the columns (93 of 189), and 0.90/0.49 ≈ 1.8. So each LF column is worth about 1.8 times the average. ×1 means that a family weighs exactly in proportion to its size. Density should be read together with the value: A with LogReg has ×3.3 but a drop of only 0.055, so it is dense because it is small.

Two results stand out. On `local_agg_graph`, trees rely almost only on LF, while networks split between LF and AF (Figure 24 in the appendix adds the spread over the seeds). On `transparent`, D is the heaviest family for all seven models (0.132 to 0.406), while structure matters mostly to the networks.

The order of the families does not depend on the period. Shuffling on validation gives the same order as on the test on all four menus with several families (for example D, A, B, C on `transparent`), so the conclusions about families are not tied to the test period. The drops are mostly smaller on the test, which is expected since the PR-AUC starts lower.

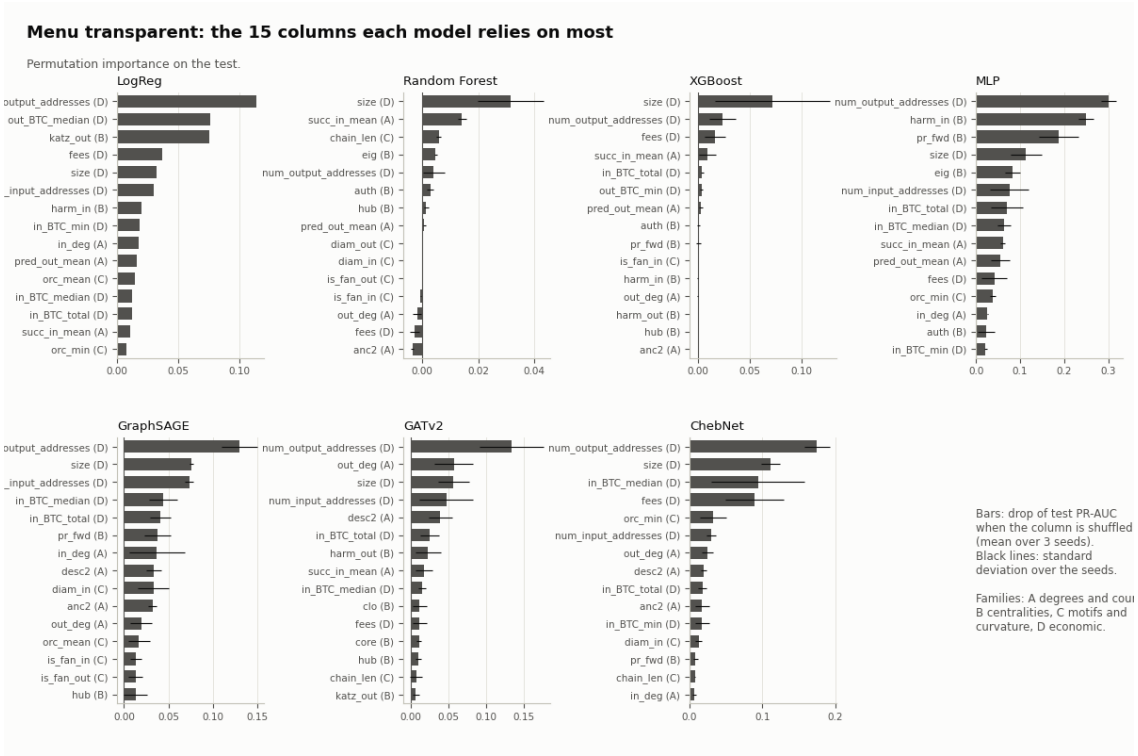


Figure 14: Menu `transparent`: the 15 columns each model relies on most. Bars: drop of test PR-AUC when the column is shuffled, mean over 3 seeds. Black lines: standard deviation over the seeds.

Columns and families. Figure 14 uses the same measure as Figure 13 at the level of single columns. The idea is the same, with one important difference: the drops of the columns do not add up to the drop of the family. With Random Forest, all of D gives 0.365, while the sum of its columns shuffled one by one is −0.044. With XGBoost the two numbers are 0.352 and 0.088, and with the MLP 0.406 and 0.679. For the trees the reason is substitution: if only `size` is ruined, the model falls back on similar columns and loses little. If all of D is ruined, no replacement is left. So the heatmap tells which *kind* of information a model needs, and Figure 14 tells which single column cannot be replaced. A small bar can mean information that is present twice.

8.3 In which direction do the columns push?

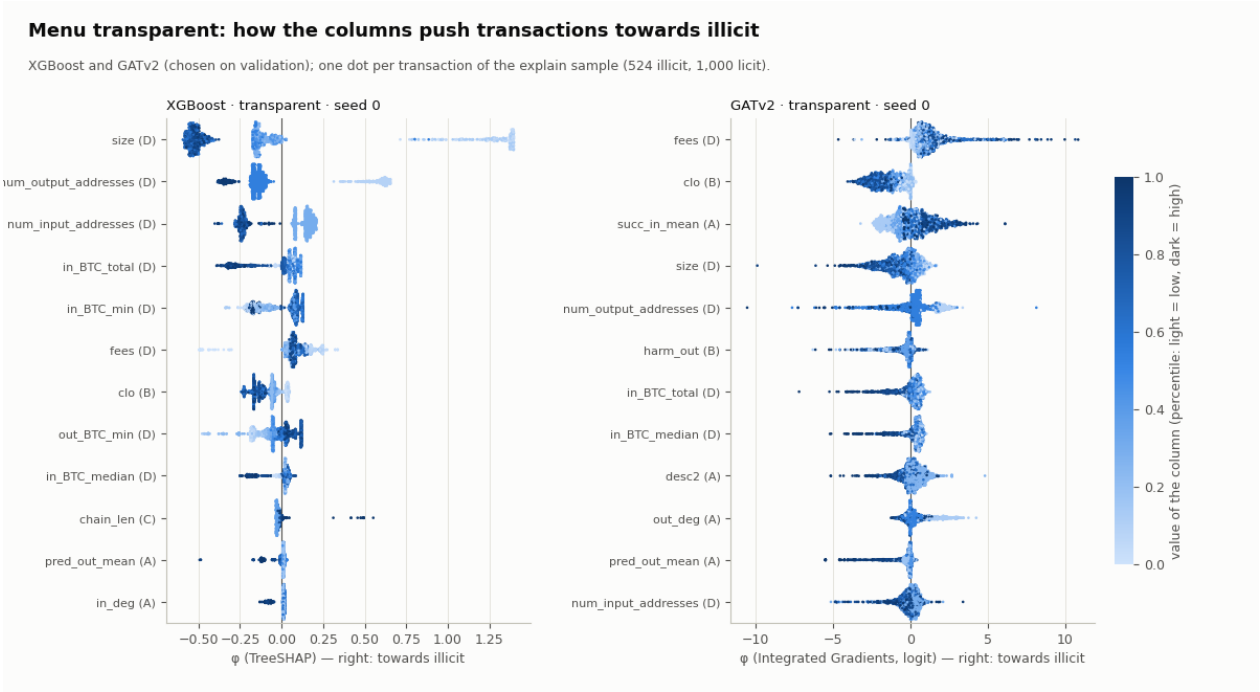


Figure 15: Menu `transparent`, seed 0. One dot per transaction of the explain sample. Position: ϕ (right pushes towards illicit). Colour: value of the column (light is low, dark is high). XGBoost is fixed in advance and GATv2 is the GNN chosen on validation.

Which models. We show XGBoost and one GNN. XGBoost is fixed in advance: it represents the trees and TreeSHAP gives exact values. The GNN is the one with the best mean validation PR-AUC: GATv2 on `transparent`, by a small margin (0.685 against 0.676 for ChebNet). XGBoost is the best model on validation and GATv2 the best GNN on validation. The test gives another order: the MLP leads (0.486), followed by Random Forest (0.442) and XGBoost (0.422), and ChebNet (0.354) beats GATv2 (0.262). We choose on validation because choosing on the test would select the model that was also lucky on the test. For transparency, the best GNN on the test has its plot in the appendix.

How to read a beeswarm. A dot carries two separate pieces of information. Position gives the direction and colour gives the value. So a negative ϕ does not mean a low value of the column. For `size` in XGBoost, the dark dots on the left ($\phi \approx -0.5$) are *high* values that push towards licit. The small light cloud on the right (ϕ up to $+1.4$) is made of *low* values that push towards illicit. Many dots on the left only reflect that most transactions in the sample are licit and larger.

The rule of XGBoost on size is a threshold. Under 200 bytes ϕ is about $+1.38$, between 200 and 300 bytes about -0.15, and above 300 bytes about -0.53. Of the 332 test transactions under 200 bytes, 78% are illicit. This rule describes the *model*, and whether small size has anything to do with crime is another question. Section 7 shows that the rule stops working after the closure.

One transaction. Figure 16 is an example chosen by a fixed rule. Among the illicit transactions of the sample flagged by XGBoost at its validation threshold (261 of 524, threshold 0.148), we take the one with the median score, so it is a typical case. One transaction is enough because we are querying a model that is already trained, and we can query it as often as we want. TreeSHAP does the following:

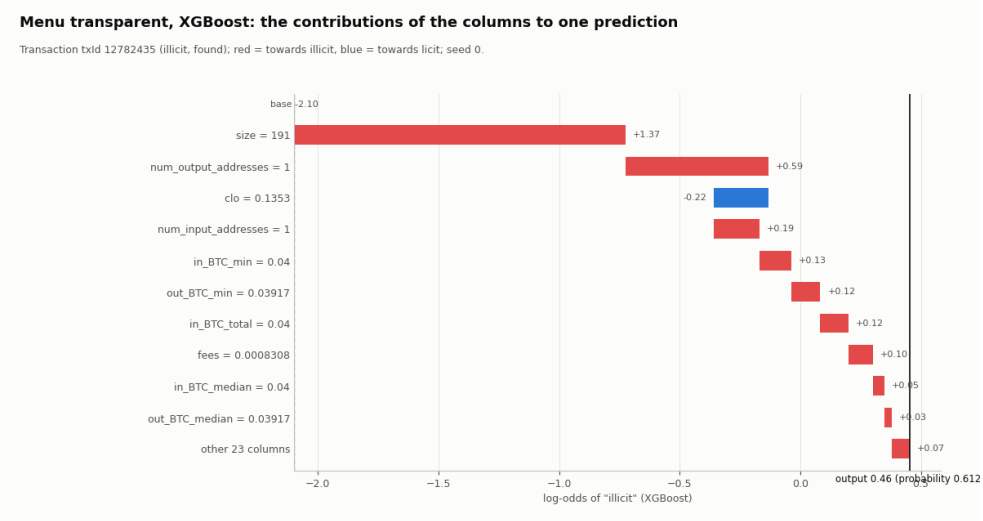


Figure 16: One transaction explained: XGBoost on `transparent`, seed 0. Red pushes towards illicit and blue towards licit. Log-odds on the horizontal axis.

1. It starts from the base value, the mean output of the model: -2.10 in log-odds, or a probability of about 0.11.
2. It asks what the model would say if it knew only some columns of this transaction. For an unknown column, the tree follows both branches, weighted by the share of train rows that went each way. So the comparison with other transactions is already inside the trees.
3. The contribution of a column is how much the output changes when the column is revealed, averaged over all the orders in which columns can be revealed. This is the Shapley value, and on trees it can be computed exactly without trying every order.
4. The pieces add up exactly to the output: from -2.10 to 0.46, a probability of 0.612.

Here `size` $= 191$ moves the output by $+1.37$, more than the sum of all other columns ($+1.18$). This explains why the *model* gave this score, and it says nothing about why the transaction is illicit.

Why only some beeswarms. A beeswarm needs columns with a name: on `local` it would say that `Local_feature_53` pushes towards illicit, which explains nothing. So `transparent` is the main plot, `econ` and `graph` are in the appendix, `local`, `local_agg` and `local_agg_graph` are excluded because LF and AF are anonymous, and `none` has no column. Each panel shows the 12 columns with the highest mean $|\phi|$ for that model.

ChebNet looks flat because of its axis. In Figure 25 in the appendix, the features of ChebNet seem to be all at zero. This is an effect of the scale. The axis goes from -80 to $+10$ because of a few extreme values: the lowest is -82 on `fees`, and 25 transactions are below -20. Table 14 shows that the typical $|\phi|$ of ChebNet is larger than that of GATv2. A small ϕ also does not mean a poor model: XGBoost has the smallest ϕ and the best PR-AUC of the three. The size of ϕ does not measure quality, for two reasons. The scale depends on the model, so between models we compare shares. And ϕ says how much the score moves, whether or not it moves in the right direction: the extreme value of -82 is on an *illicit* transaction, to which ChebNet gives a logit of -73, so it is very confident and wrong.

8.4 Own columns or neighbours?

In Figure 17, the value 0.80 for family B on `graph` with GraphSAGE means that of all the attribution given to the B columns, 80% comes from the values of those columns in the *neighbours* and 20%

Table 14: Size of the attributions on `transparent`: seed 0, all 33 columns, explain sample. PR-AUC: mean over the seeds.

Model	Median $\lvert\phi\rvert$	90th percentile	Maximum	Test PR-AUC
ChebNet	0.34	1.76	82.4	0.354
GATv2	0.18	1.12	11.4	0.262
XGBoost	0.01	0.15	1.4	0.422

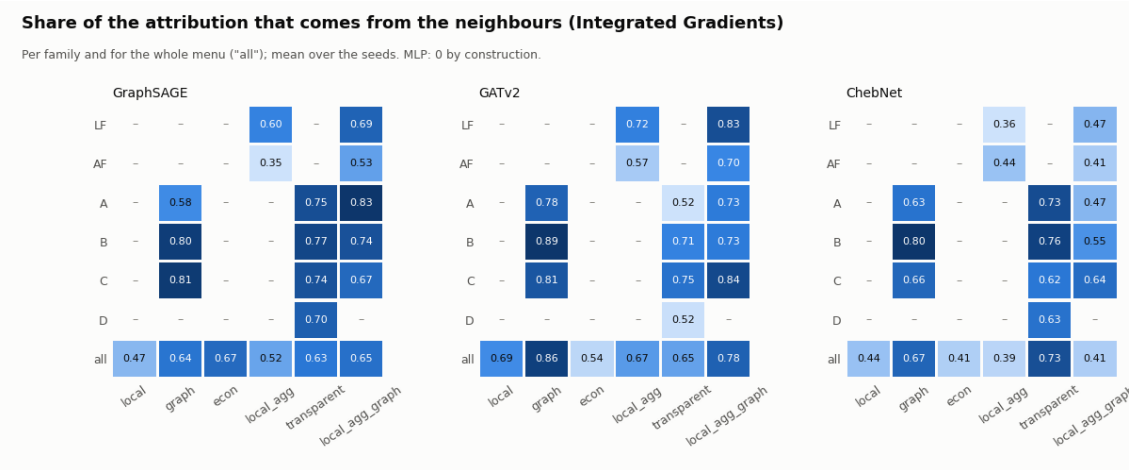


Figure 17: Share of the Integrated Gradients attribution that comes from the neighbours, per family and for the whole menu ("all"). The attributions of the GNNs come from seed 0 only, so the "mean over the seeds" in the subtitle of the plot is that one seed. The MLP is 0 by construction.

from the transaction *itself*:

$$\text{cell} = \frac{|\text{neighbour part}|}{|\text{own part}| + |\text{neighbour part}|}.$$

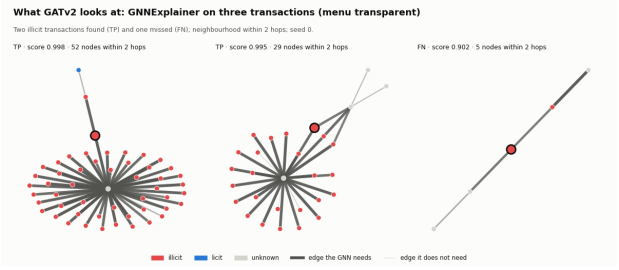
The columns of the map do not need to add up to 1, because each cell is a fraction on its own, and its 100% is own plus neighbours. What adds up to 1 is a different measure, the share of each family in the total: for GraphSAGE on `graph`, A has 0.21, B has 0.57 and C has 0.22. The row "all" is the same fraction for the whole menu, computed after summing the columns. It is 0.64 while a weighted mean of the rows would be about 0.76, because signs cancel in the sum.

On `graph` alone, the GNNs take most of their attribution from the neighbours (0.64 to 0.86). This fits their gain over the MLP on this menu.

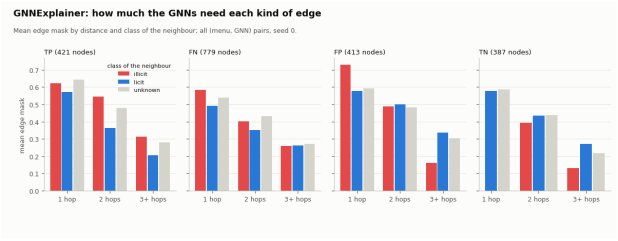
8.5 Which edges do the GNNs need?

How GNNExplainer works. Take a trained model and one transaction, and put a *dimmer* on each edge of its neighbourhood, from 0 (off) to 1 (on), that multiplies the message of that edge. For 200 steps the method adjusts the dimmers with two opposing goals: the score of the transaction must stay at its original value, and each edge left on has a cost. At the end a high mask means that the score changes without that edge, so the model needs it, and a low mask means that the edge can be switched off.

We ask it to keep the *logit* and we use the regression mode of the method. Keeping the predicted class does not work on this data: on a transaction predicted with confidence, the loss is already near 0 and turning off an edge changes nothing, so the masks stay close to their random start. We explain 100 nodes per (menu, GNN) pair on seed 0: a common core of 60 illicit and 20 licit transactions, so that the 20 pairs can be compared, and 20 false positives of each model, since these are the errors we most want to understand. This gives 2,000 explanations. Averages use only the edges whose message can reach the node.



(a) Three transactions, GATv2 on `transparent`, seed 0. Thick and dark edges are the ones the GNN needs.



(b) Mean mask by distance and class of the neighbour, by outcome of the prediction, all 20 pairs, seed 0.

Figure 18: GNNExplainer: examples and averages.

Reading the bars. In Figure 18b, a high value for illicit neighbours at 1 hop means that the GNN needs the messages from its direct illicit neighbours to reproduce the score. A low value means that those edges can be switched off. A mask measures dependence and gives no direction, so an edge that lowers the score can also have a high mask. Masks start around 0.5, so the bars should be compared with each other, and 0.6 is not "much" in itself.

- The cleanest signal is in true positives at 2 hops: illicit 0.55, unknown 0.48, licit 0.37. At 1 hop the bars are close (0.62, 0.64, 0.57), and illicit beats licit in only 11 of 18 pairs.
- At 3 or more hops, illicit neighbours are the highest for true positives (0.31) and the lowest for false positives and true negatives (0.16 and 0.13).
- Masks fall with distance, as expected.
- The tallest bar, 0.73 for false positives at 1 hop, comes from only 6 nodes and 12 edges. True negatives have no red bar at 1 hop because none of them has a direct illicit neighbour in the sample.

Explaining again with another seed of the explainer gives a mean Spearman of 0.57 and an overlap of the 10 strongest edges of 0.74. With a model from another seed the values are 0.62 and 0.52. So small differences between bars should not be overread.

Do true positives simply have more neighbours? In Figure 18a the two true positives have many nodes and the false negative has few, which may suggest that GATv2 relies on the number of neighbours. The plot is only an example: three nodes chosen by a fixed rule (among core nodes with 5 to 60 nodes within 2 hops, so that they can be drawn, the two true positives and the false negative with the highest score). On all the data the idea does not hold. The median number of direct neighbours is 1 for both true positives and false negatives, on all 524 illicit transactions and in every seed. The Spearman correlation between score and number of neighbours on illicit transactions is 0.02 to 0.06. In the core, the medians are close (13 nodes for true positives, 9.5 for false negatives), the largest neighbourhoods belong to false negatives (154 nodes against 52), and those over 60 nodes are not drawn. The many nodes of the true positives come from a single unknown hub, the centre of the star. The false negative has a score of 0.902 and it counts as missed only because the threshold is 0.914.

8.6 How stable are the explanations?

Each cell of Figure 19 compares two rankings of the columns by importance. The three maps differ in what is compared:

- **Between seeds:** the same model and menu trained with different seeds. A high value means that the ranking does not depend on chance (mean 0.78). MLP on `econ` is low (0.20). LogReg is 1.00 because it is deterministic.

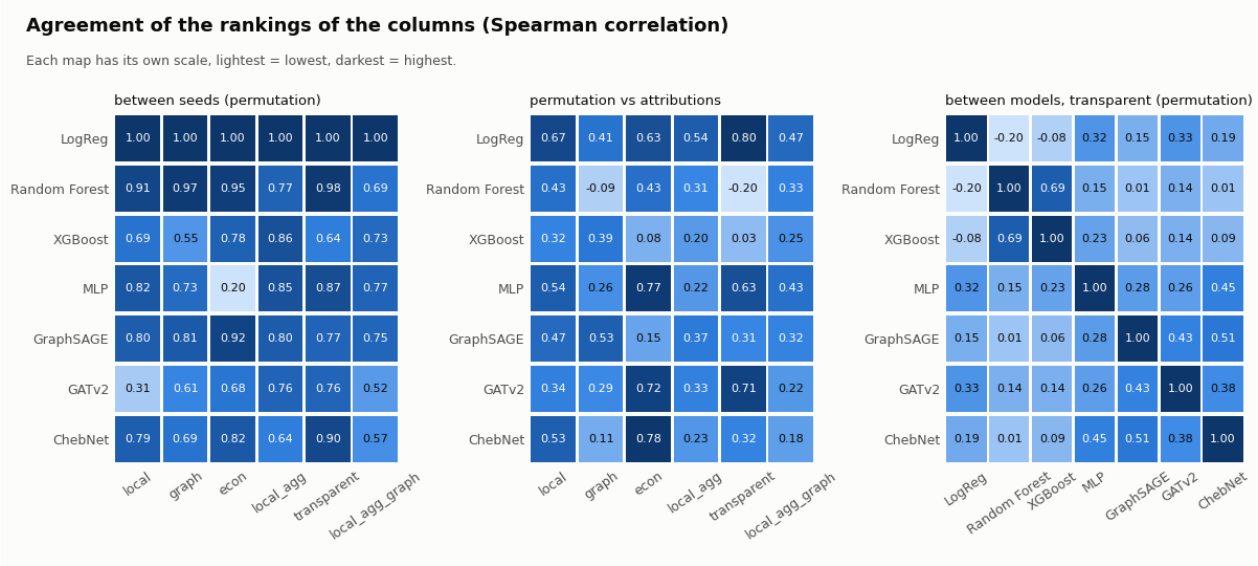


Figure 19: Agreement between rankings of the columns (Spearman correlation). 1 means the same order, 0 means no relation, and a negative value means a tendency to the opposite order. Each map has its own colour scale, so compare the numbers.

- **Permutation against attributions:** two ways of measuring importance on the same model. A high value means that both point to the same columns (mean 0.38). Random Forest on `transparent` is −0.20.
- **Between models:** two models on `transparent`. A high value means that they use the same columns (mean 0.33). The two tree models agree (0.69), while LogReg and Random Forest do not (−0.20).

A low value is not always an error. Permutation measures the drop of a metric and attributions measure the movement of the score, so if two columns replace each other, permutation gives both about 0 while SHAP splits the credit. Spearman also counts the whole ranking: XGBoost on `transparent` has 18 of 33 columns with a drop near 0, and the order among them is noise, which pushes the correlation down to 0.03 even though the 10 top columns overlap by 50%. On `econ` there are only 9 columns, so little is needed to move the value.

The message for this report is that conclusions about *families*, such as "D dominates on `transparent`", are solid, while rankings of *single columns* depend on the model and on the method.

9 Discussion

Are opaque features needed? Before the closure, far less than the numbers on the whole test set suggest: 33 named columns reach 0.79–0.81 against 0.91, and 9 economic columns reach 0.78 with XGBoost. Named features are clearly weaker only at the very top of the ranked list, where about two of every three flags are correct in the first 100, against all of them. Since family D largely repeats LF, this result also says that a large part of the value of LF lies in a few facts that anyone can read from the blockchain: size, number of inputs and outputs, amounts and fees.

How useful is the structure? The remark in [3] holds, and now it has numbers: with structure alone, the PR-AUC is 0.13–0.30 against 0.65. The picture changes with the economic columns: `transparent` is well above `graph`. Structure gives little on top of other features, but on structure alone two of the three GNNs clearly beat the MLP, and it may hold up better after the closure.

Do GNNs help? They help a neural network on some menus, and they do what we expect when they do (Figure 10). They do not reach trees when the local features are available.

The value of interpretability. With anonymous features we could only have said that the models fail after time step 43. With named features we can read why: the models learned that small, simple transactions are illicit, and after the closure illicit transactions are larger than licit ones. So interpretability shows both what a model knows and when its knowledge expires. For a real detector this is the main warning of this work: high scores on validation (0.96) say little if the behaviour of criminals changes.

Limits.

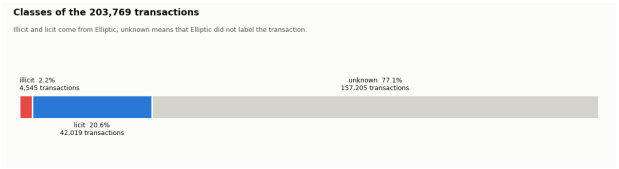
- We use a single temporal split, and the test window is dominated by two time steps: 355 of 524 illicit transactions are in time steps 41–42.
- The after period has only 169 illicit transactions, and some time steps have very few.
- Three seeds are few, as GATv2 on `local` shows (0.425, 0.214, 0.510).
- The labels of the dataset are themselves the result of a process we cannot inspect, and 77% of the transactions have none.
- The link between the fall at time step 43 and the closure of the market comes from [1]. We observe a change in the data, and we did not verify its cause.
- Models were never retrained after the closure. A detector retrained over time could behave differently, though [1] report that even a Random Forest retrained at each time step does not solve this.
- Integrated Gradients on the GNNs rests on one seed, and GNNExplainer on seed 0 with moderate stability.

Next steps. The results suggest three directions. First, study which features stay stable across time, starting from family C and from the structural signal of GATv2. Second, repeat the comparison with several temporal splits. Third, retrain periodically and measure how fast the models recover after the closure.

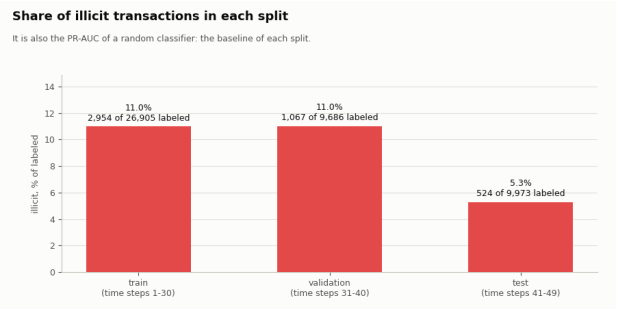
10 Conclusion

We replaced the opaque features of Elliptic with 33 features that have a clear meaning, and we measured the result on 132 final runs. Named features keep most of the performance before the closure of the dark market (0.79–0.81 against 0.91), and for trees almost all of it comes from a few economic columns. The graph alone is weak and adds little to the opaque features, and trees without edges beat every GNN when local features are available. After the closure, no feature set works, because illicit transactions change shape. The named features let us see this change and read the rule that stops working.

A Additional figures



(a) Classes of the 203,769 transactions.



(b) Share of illicit among labelled transactions, per split.

Figure 20: Classes and splits. The values in (b) use all labelled transactions, while Table 4 uses the evaluation rows.

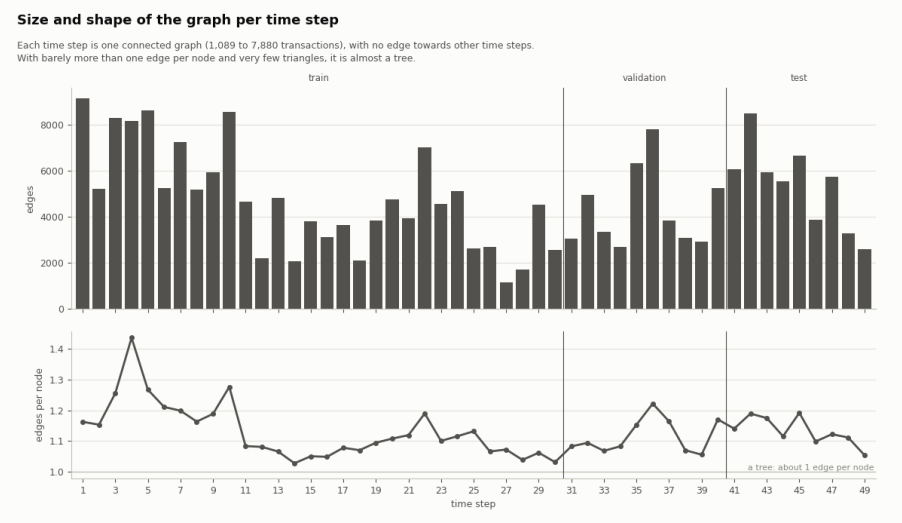
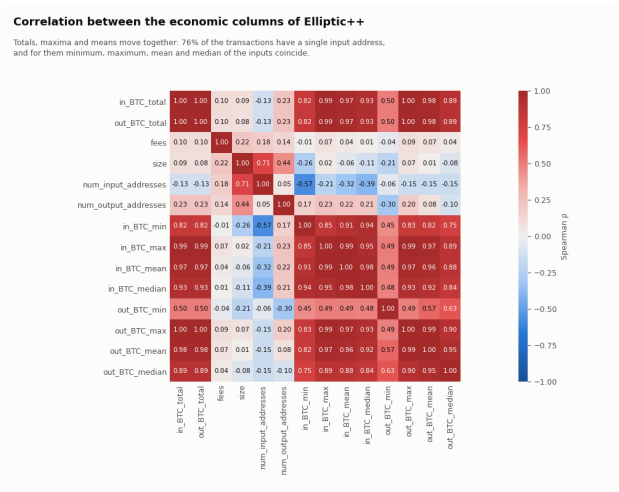
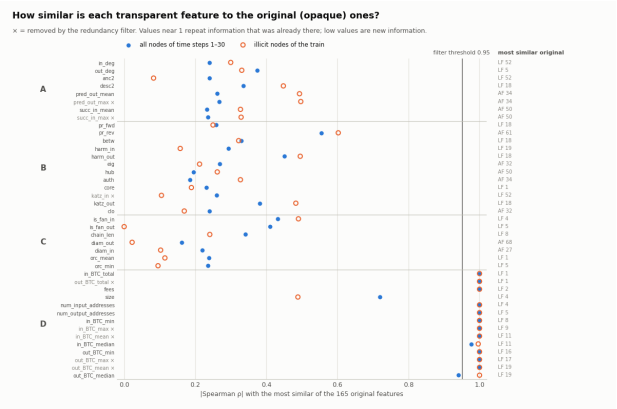


Figure 21: Edges per time step, and edges per node. With barely more than one edge per node and very few triangles, the graph is almost a tree.



(a) Correlation between the 14 economic columns.



(b) Each transparent feature against its most similar opaque one. Values near 1 repeat known information, and low values are new.

Figure 22: More correlations. Family D is close to the opaque features, and families A, B and C are far from them.

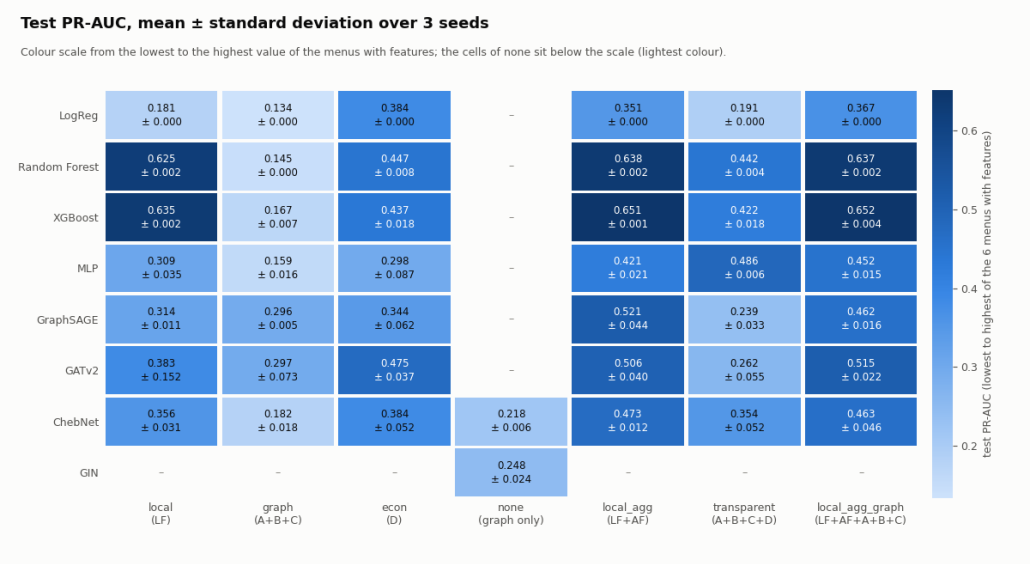


Figure 23: Test PR-AUC, mean ± standard deviation over 3 seeds. The colour scale runs from the lowest to the highest value of the six menus with features.

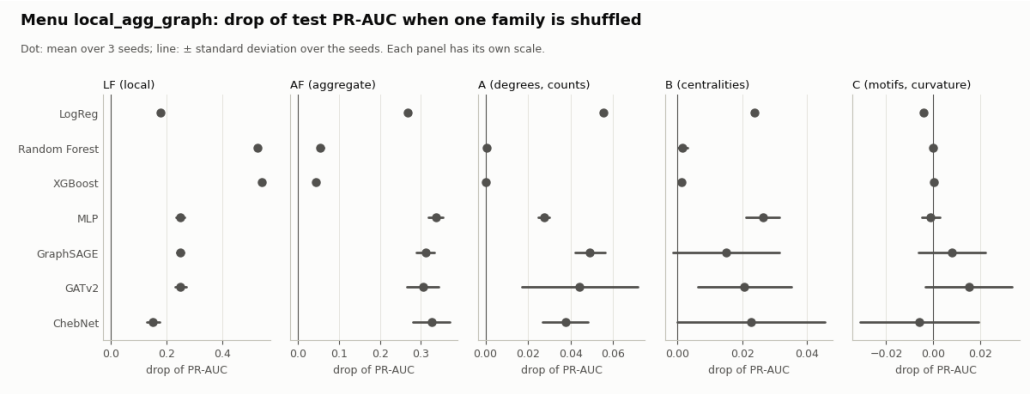


Figure 24: Menu `local_agg_graph`: drop of test PR-AUC when one family is shuffled. Dot: mean over 3 seeds. Line: ± one standard deviation. Each panel has its own scale.

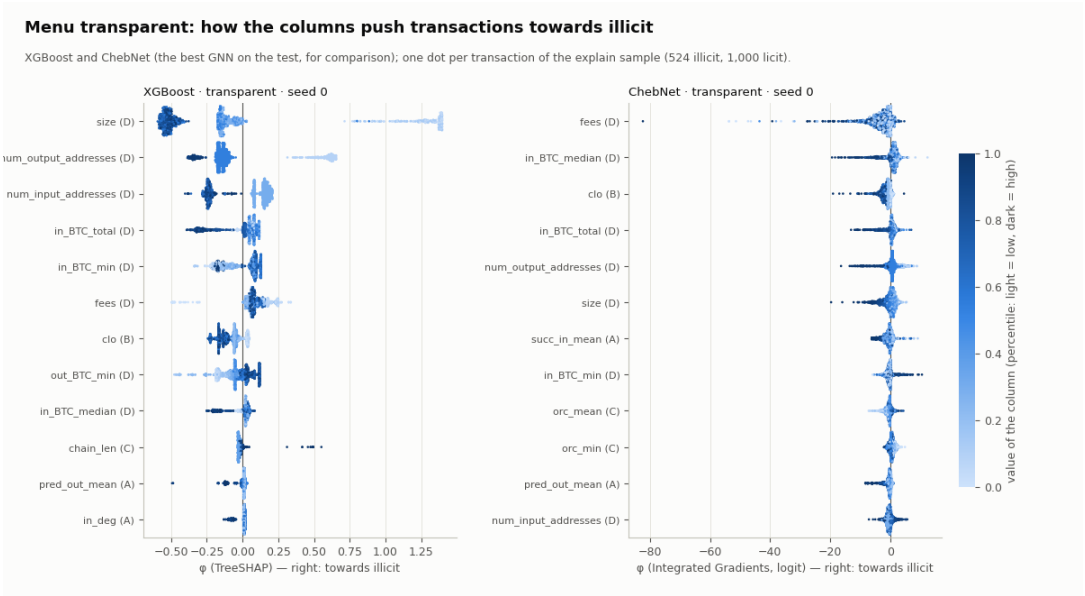
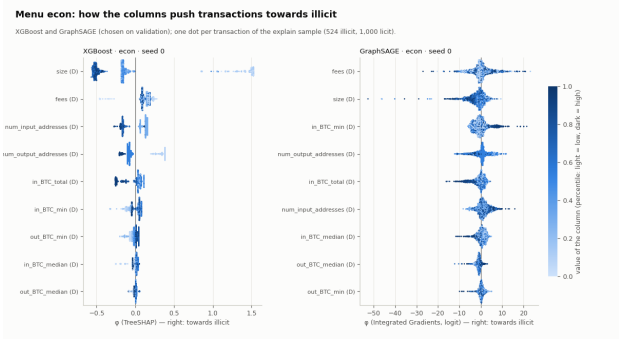
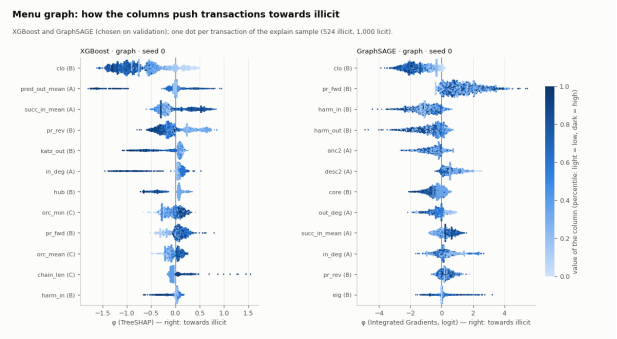


Figure 25: Menu `transparent`: XGBoost and ChebNet, the best GNN on the test, shown for comparison. The axis of ChebNet is stretched by a few extreme values (Section 8.3).



(a) Menu `econ`: XGBoost and GraphSAGE.



(b) Menu `graph`: XGBoost and GraphSAGE.

Figure 26: The beeswarm plot on the two groups taken alone, seed 0. The GNN is the one chosen on validation.

B Search spaces

Table 15: Hyperparameters searched for each model. All neural models also tune the learning rate (10^{-4} to 10^{-2}, log scale) and the weight decay (10^{-6} to 10^{-2}, log scale).

Model	Search space
LogReg	C: 10 values from 10^{-4} to 100; class weight: none or balanced. The grid of 20 combinations is fully explored.
Random Forest	500 trees; max features: sqrt, 0.1, 0.2, 0.33, 0.5; min samples per leaf: 1, 2, 5, 10, 20; max depth: none, 10, 20; class weight: none or balanced per subsample.
XGBoost	Up to 2,000 trees with early stopping; learning rate: 0.01 to 0.3; max depth: 3 to 10; min child weight: 1 to 20; subsample: 0.5 to 1; column sample: 0.3 to 1; L2 regularization: 0.001 to 10; gamma: 0 to 5; class weight.
MLP	Layers: 1, 2, 3; hidden size: 32 to 256; dropout: 0 to 0.6.
GraphSAGE	Layers: 2, 3; hidden size: 32 to 256; aggregation: mean or max; variant: base, lin, skip; dropout: 0 to 0.6.
GATv2	Layers: 2, 3; heads: 1, 2, 4, 8; hidden size per head: 16, 32, 64; variant: base, lin, skip; dropout and attention dropout.
ChebNet	K: 2, 3, 4; layers: 2, 3; hidden size: 32 to 256; variant: base, lin, skip; dropout: 0 to 0.6.
GIN	Layers: 2 to 5; hidden size: 32 to 128; learnable ε: yes or no; jumping knowledge: none, max, concatenation; dropout: 0 to 0.5.

C Structure of the project

The work is a sequence of notebooks. Each one starts with tests on small problems with known answers, saves its results as soon as they are ready, and can resume after an interruption. Each step reads only the files of earlier steps.

Table 16: The steps of the project.

Step	Content
00	Data exploration: classes, splits, graph properties, economic columns, correlations.
01	Computation of the 27 graph features.
02	Cleaning, redundancy filter, menus, transformations, configuration file.
03	Timing test and tuning of the 44 pairs (1,480 trials).
04	Final runs: 132 triples, with scores, models, curves and bootstrap saved.
05	Tables and plots of the comparison between menus.
06	Permutation importance, attributions, GNNExplainer, and their plots.
07	Analysis of the closure of the dark market, from the saved scores.

References

[1] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. In *KDD Workshop on Anomaly Detection in Finance*, 2019.

[2] Y. Elmougy and L. Liu. Demystifying Fraudulent Transactions and Illicit Nodes in the Bitcoin Network for Financial Forensics. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023.

[3] S. Ferretti, G. D'Angelo, and V. Ghini. Enhancing Anti-Money Laundering Frameworks: An Application of Graph Neural Networks in Cryptocurrency Transaction Classification. *IEEE Access*, vol. 13, 2025.

[4] S. Ferretti, G. D'Angelo, and V. Ghini. On the Use of Heterogeneous Graph Neural Networks for Detecting Malicious Activities: a Case Study with Cryptocurrencies. In *4th International Workshop on Open Challenges in Online Social Networks (OASIS)*, ACM, 2024.